

```
In [5]: import numpy as np
import pandas as pd

import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [np, pd, mpl, sns]:
    print(i.__name__, i.__version__)
```

```
numpy 1.26.4
pandas 2.1.4
matplotlib 3.8.0
seaborn 0.12.2
```

3-2 일반화 (Generalization)

일반화란 모델이 학습 데이터뿐만 아니라 새로운 데이터에서도 잘 작동하는 것을 의미합니다.

머신러닝 모델이 실제 상황에서 높은 성능을 가지게 하려면 일반화가 잘 되어 있어야 합니다.

머신러닝의 궁극의 목표는 **일반화 성능(Generalized Performance)** 향상입니다.

```
In [2]: Image('123.png')
```

Out[2]:

0. 데이터셋 소개

Wild blueberry

[Wild blueberry](#) 야생 블루베리의 수확량을 시뮬레이션한 데이터셋입니다.

Features	Unit	Description
Row#		Index
clonesize	m2	The average blueberry clone size in the field
honeybee	bees/m2/min	Honeybee density in the field
bumbles	bees/m2/min	Bumblebee density in the field
andrena	bees/m2/min	Andrena bee density in the field
osmia	bees/m2/min	Osmia bee density in the field
MaxOfUpperTRange	°C	The highest record of the upper band daily air temperature during the bloom season
MinOfUpperTRange	°C	The lowest record of the upper band daily air temperature
AverageOfUpperTRange	°C	The average of the upper band daily air temperature
MaxOfLowerTRange	°C	The highest record of the lower band daily air temperature
MinOfLowerTRange	°C	The lowest record of the lower band daily air temperature
AverageOfLowerTRange	°C	The average of the lower band daily air temperature
RainingDays	Day	The total number of days during the bloom season, each of which has precipitation larger than zero
AverageRainingDays	Day	The average of raining days of the entire bloom season
fruitset		The amount of fruit per tree
fruitmass		The mass of a fruit
seeds		The mass of a seed
yield		The amount of yield

간단하게 여러 아이디어를 실험해볼 수 있는 데이터셋입니다.

```
In [6]: df_berry = pd.read_csv("data/wild_blueberry.csv", index_col='Row#')
df_berry.head()
```

```
Out[6]:
```

	clonesize	honeybee	bumbles	andrena	osmia	MaxOfUpperTRange	MinOfUpperTRange	AverageOfUpperTRange	MaxOfLowerTF
Row#									
0	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
1	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
2	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
3	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
4	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	

```
In [ ]:
```

1. 편향(Bias), 분산(Variance)

머신러닝모델의 편향(Bias), 분산(Variance)

편향(Bias): 머신러닝 모델의 출력이 치우친 정도를 의미합니다.

- 높은 편향(High Bias)는 모델 출력의 쓸림이 큰 상태
- 낮은 편향(Low Bias)는 모델 출력이 쓸림이 낮은 상태

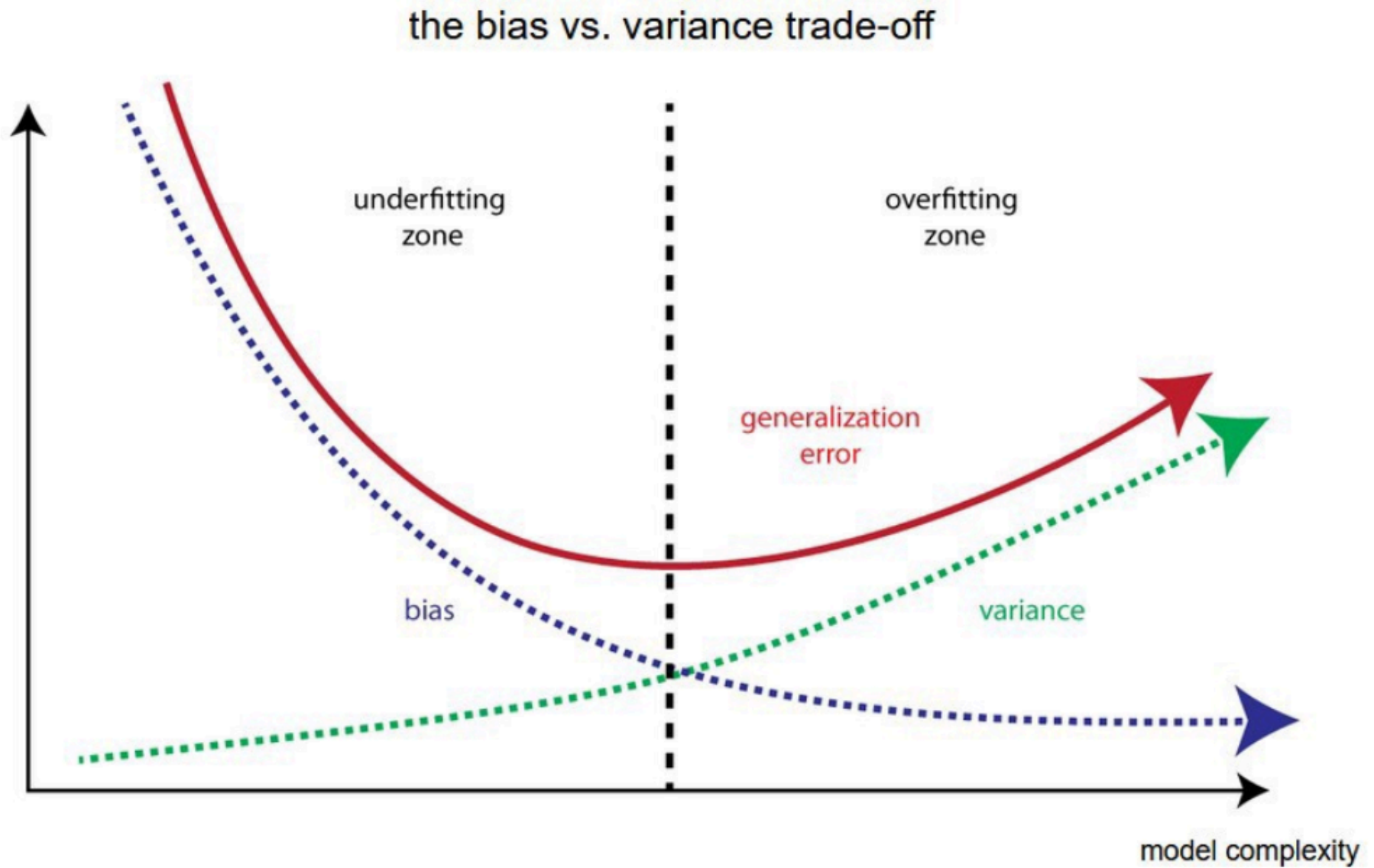
분산(Variance): 머신러닝 모델의 출력이 흩어진 정도를 나타하며,

데이터에 얼마나 민감하게 반응하는지를 의미합니다.

- 높은 분산(High Variance)은 모델 출력의 편차가 큰 상태
- 낮은 분산(Low Variance)는 모델 출력의 편차가 낮은 상태

```
In [3]: Image('124.png')
```

```
Out[3]:
```



편향과 분산에 영향을 주는 요소

1. 모델의 복잡도: 단일 측정 요소(estimator)가 출력을 도출하기 위한 요소의 수

입력 변수의 수, Parameter의 수, 의사 결정 나무의 노드수, 지지 벡터(Support vector)의 수

→ 모델의 복잡도를 높힐 수록 편향이 낮아지는 경향이 있고, 분산은 높아지는 경향이 있습니다.

2. 모델의 측정 요소(estimator)의 수: 모델의 출력을 집계 내기 위해 할당된 측정 요소의 수

Random Forest에서 의사결정나무의 수, 최근접이웃의 수

→ 모델의 측정 요소를 늘릴 수록 분산은 낮아지는 경향이 있습니다.

3. 파라미터의 절대 크기(Magnitude)

선형 회귀 계수의 크기, 인공 신경망의 가중치 크기

→ 파라미터의 절대 크기가 클수록 분산이 높아지는 경향이 있습니다.

2. 과적합(Overfitting)과 과소적합(Underfitting)

1. **과적합(Overfitting)**: 모델이 학습데이터에 지나치게 맞추어져서 일반화 성능이 떨어진 상태입니다.

- 분산이 높거나 편향이 낮을 경우에 쉽게 발생합니다.
- 데이터가 지닌 패턴과 모델이 나타내는 패턴의 차이가 클수록 쉽게 발생합니다.

▷ 모델의 설정을 바꾸어 데이터에 보다 적합한 형태로 바꿉니다.

▷ 모델에 맞게 데이터를 바꿉니다.

- 학습데이터와 평가데이터의 차이가 클 경우 쉽게 발생합니다.

▷ 속성 공학을 통해 학습데이터와 평가데이터의 패턴을 반영할 수 있는 형태의 변수들을 만들어 줍니다.

- 데이터가 지닌 특성(패턴)을 나타내기에 데이터 수가 충분하지 않을 경우 쉽게 발생합니다.

▷ 데이터의 수를 늘립니다.

▷ 모델의 복잡도를 낮추어 봅니다.

- 부적절한 입력 변수가 포함이 되어 있을 경우 쉽게 발생합니다.

▷ 속성 선택을 통해 적절한 변수를 뽑아냅니다.

- 이상치나 노이즈에 의해 데이터에 오염이 심할수록 쉽게 발생합니다.

▷ 데이터 클린징 통해 이상치와 노이즈의 영향을 최소화 합니다.

2. **과소적합(Underfitting)**: 모델이 학습데이터에 맞추어지지 않아 일반화 성능이 떨어진 상태입니다.

- 분산이 낮거나 편향이 클 경우에 쉽게 발생합니다.

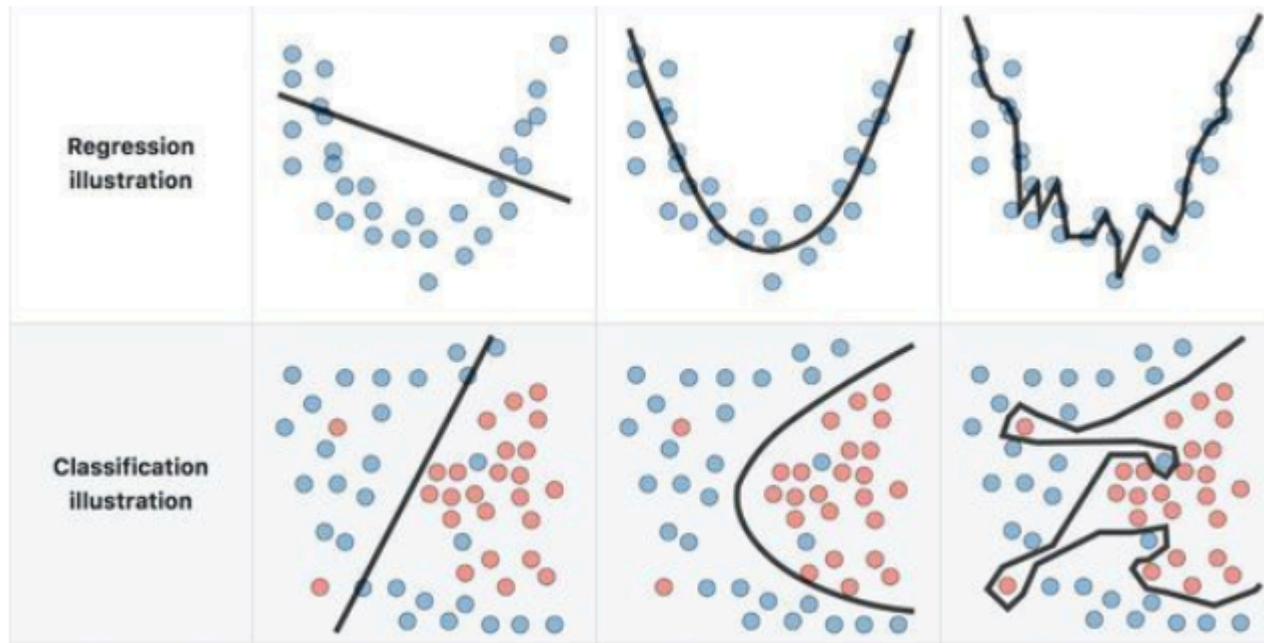
- 모델의 손실을 낮추는 최적화 과정이 제대로 이루어지지 않았을 경우 발생합니다.

▷ 학습 과정을 추가적으로 진행하여 모델을 좀 더 학습데이터에 맞춥니다.

- 모델의 복잡도를 높혀 학습 데이터를 좀 더 반영할 수 있도록 합니다.

In [4]: `Image('125.png')`

Out[4]:

**[Ex.1]**

K-최근접 이웃모델의 근접이웃의 수가 많을 수록 편향은 높아지고, 분산은 낮아집니다.

df_berry 에서 yield를 예측하는 K-최근접 이웃모델의 근접이웃 수에 따른 분산의 변화를 살펴봅니다.

전체 데이터를 80% 학습 데이터 df_train, 20%는 평가 데이터 df_test로 나눕니다.

이에 따른 학습데이터의 지표와 평가데이터의 지표를 살펴 봅니다.

입력 변수는 정규화(Normalization, Min-Max Scaling)를 합니다.

성능 지표는: 평균제곱오차(MSE, Mean Squared Error)를 사용합니다.

```
In [7]: from sklearn.model_selection import train_test_split
        from sklearn.metrics import mean_squared_error
```

```
# 80%는 학습 20%는 평가셋으로 나눕니다.
df_train, df_test = train_test_split(df_berry, train_size=0.8,
                                     random_state=123)
```

In [4]: `from sklearn.preprocessing import MinMaxScaler`
`from sklearn.neighbors import KNeighborsRegressor`

...

코드는 Python에서 데이터프레임(df_berry)의 열(column) 이름 중 'yield'를 제외한 모든 열 이름을 X_cols라는 리스트에 할당합니다.

여기서 사용된 코드:

1. 리스트 컴프리헨션(List Comprehension):

- ``[i for i in df_berry.columns if i != 'yield']``
- ``df_berry.columns``는 데이터프레임(df_berry)의 모든 열 이름을 반환합니다.
- ``for i in df_berry.columns``는 데이터프레임의 각 열 이름을 순회합니다.
- ``if i != 'yield'``는 열 이름이 'yield'가 아닌 경우에만 해당 열 이름을 가져옵니다.
- 따라서, ``i``가 'yield'가 아닌 모든 열 이름을 리스트에 추가합니다.

2. 할당: ``X_cols = [...]'``

- 리스트 컴프리헨션의 결과를 ``X_cols``에 할당합니다.

코드를 실행하면 'yield' 열을 제외한 모든 열 이름이 X_cols 리스트에 저장되어 있게 됩니다.
 이후 이 리스트를 분석이나 모델링에 사용할 수 있습니다.

...

입력 변수 리스트를 만듭니다.

```
X_cols = [i for i in df_berry.columns if i != 'yield']
```

MinMax Scaler를 생성합니다.

```
mm_scaler = MinMaxScaler()
```

MinMax Scaler를 학습 데이터로 학습시킵니다.

```
mm_scaler.fit(df_train[X_cols])
```

```
# 학습 데이터의 입력 변수를 변환하여 X_train 디자인 행렬을 만듭니다.
X_train = mm_scaler.transform(df_train[X_cols])
y_train = df_train['yield']

# 평가 데이터의 입력 변수를 변환하여 X_test 디자인 행렬을 만듭니다.
X_test = mm_scaler.transform(df_test[X_cols])
y_test = df_test['yield']
```

In [5]:

```
'''
코드는 K 최근접 이웃(K-Nearest Neighbors, KNN) 회귀 모델을
사용하여 학습 및 평가를 수행하고,
그 결과를 기록하는 과정을 보여줍니다.

1. `v`, `mse_train`, `mse_test` 리스트 초기화:
- `v`, `mse_train`, `mse_test`는 각각 출력의 분산,
  학습 데이터에 대한 평균 제곱 오차(MSE),
  평가 데이터에 대한 MSE를 저장할 리스트입니다.
  초기화를 위해 `list()`를 사용하여 빈 리스트를 생성합니다.

2. 반복문:
- `for i in range(1, 31):`를 통해 최소 이웃의 수가 1에서
  30까지 변경되는 반복문을 생성합니다.

3. KNN 모델 학습:
- `reg_knn = KNeighborsRegressor(n_neighbors=i)`를 통해
  최근접 이웃의 수를 `i`로 설정한 KNN 회귀 모델을 생성합니다.
- `reg_knn.fit(X_train, y_train)`을 통해 훈련 데이터(X_train)와
  타겟 변수(y_train)를 사용하여 모델을 학습합니다.

4. 예측 및 성능 측정:
- `y_train_hat = reg_knn.predict(X_train)`을 사용하여
  학습 데이터에 대한 예측값을 계산합니다.
- `np.var(y_train_hat)`을 사용하여 모델의 학습 데이터에 대한
  출력의 분산을 계산하고, 이를 `v` 리스트에 추가합니다.
- `mean_squared_error(y_train, y_train_hat)`을 사용하여
  학습 데이터에 대한 평균 제곱 오차(MSE)를 계산하고,
  이를 `mse_train` 리스트에 추가합니다.
- `mean_squared_error(y_test, reg_knn.predict(X_test))`를
  사용하여 평가 데이터에 대한 MSE를 계산하고,
```

이를 ``mse_test`` 리스트에 추가합니다.

이 과정을 통해 반복문이 실행될 때마다 최소 이웃의 수가 변하면서
각 경우에 대한 모델의 학습 및 평가가 수행되고,
그 결과가 ``v``, ``mse_train``, ``mse_test`` 리스트에 기록됩니다.
'''

```
v = list()
mse_train, mse_test = list(), list()
# 1 ~ 30개(최소 이웃) 까지 반복시킵니다.
for i in range(1, 31):
    # 최소 이웃의 수를 i 설정하여 KNeighborRegressor를 생성합니다.
    reg_knn = KNeighborsRegressor(n_neighbors=i)
    reg_knn.fit(X_train, y_train)
    y_train_hat = reg_knn.predict(X_train)
    # 모델의 학습 데이터에 대한 출력의 분산을 기록합니다.
    v.append(np.var(y_train_hat))
    # 학습셋에 대한 성능을 기록합니다.
    mse_train.append(mean_squared_error(y_train,
                                         y_train_hat))

    # 평가셋에 대한 성능을 기록합니다.
    mse_test.append(mean_squared_error(y_test,
                                       reg_knn.predict(X_test)))
```

In [6]:

```
'''
코드는 matplotlib를 사용하여 최근접 이웃의 수에 따른
출력의 분산(Variance)과 학습 데이터 및 평가 데이터에 대한
평균 제곱 오차(MSE)를 시각화하는 그래프를 생성합니다.

1. `plt.plot(np.arange(1, len(v) + 1), v, label='Variance')`:
   - 최근접 이웃의 수에 따른 출력의 분산을 나타내는
   선 그래프를 생성합니다. x 축은 최근접 이웃의 수를 나타내고,
   y 축은 출력의 분산을 나타냅니다. 라벨은 'Variance'로 설정됩니다.
2. `plt.gca().set_xlabel('n_neighbors')`:
   - x 축의 레이블을 'n_neighbors'로 설정합니다.
   이는 최근접 이웃의 수를 나타냅니다.
3. `plt.gca().set_ylabel('Variance of $\hat{y}$')`:
   - y 축의 레이블을 'Variance of $\hat{y}$'로 설정합니다.
   $\hat{y}$는 모델의 예측값을 나타냅니다.
4. `plt.legend()`:
```

```

- 범례를 표시합니다.
5. `plt.axvline(np.argmax(mse_test) + 1)` :
- 평가 데이터에 대한 MSE가 최소가 되는 지점을
  나타내는 세로 선을 추가합니다.
6. `tax = plt.gca().twinx()` :
- 오른쪽 y 축을 생성합니다.
7. `tax.plot(np.arange(1, len(v) + 1), mse_train,
             label = 'MSE of train', color='orange')` :
- 학습 데이터에 대한 MSE를 오른쪽 y 축에 대한
  선 그래프로 생성합니다.
  라벨은 'MSE of train'으로 설정되고, 색상은 오렌지로 지정됩니다.
8. `tax.plot(np.arange(1, len(v) + 1), mse_test,
             label = 'MSE of test', color='green')` :
- 평가 데이터에 대한 MSE를 오른쪽 y 축에 대한
  선 그래프로 생성합니다.
  라벨은 'MSE of test'으로 설정되고, 색상은 녹색으로 지정됩니다.
9. `tax.set_ylabel('Mean Squared Error')` :
- 오른쪽 y 축의 레이블을 'Mean Squared Error'로 설정합니다.
10. `plt.legend()` :
- 오른쪽 y 축의 그래프에 대한 범례를 표시합니다.
11. `plt.tight_layout()` :
- 그래프 간의 충돌을 방지하기 위해 간격을 조정합니다.

```

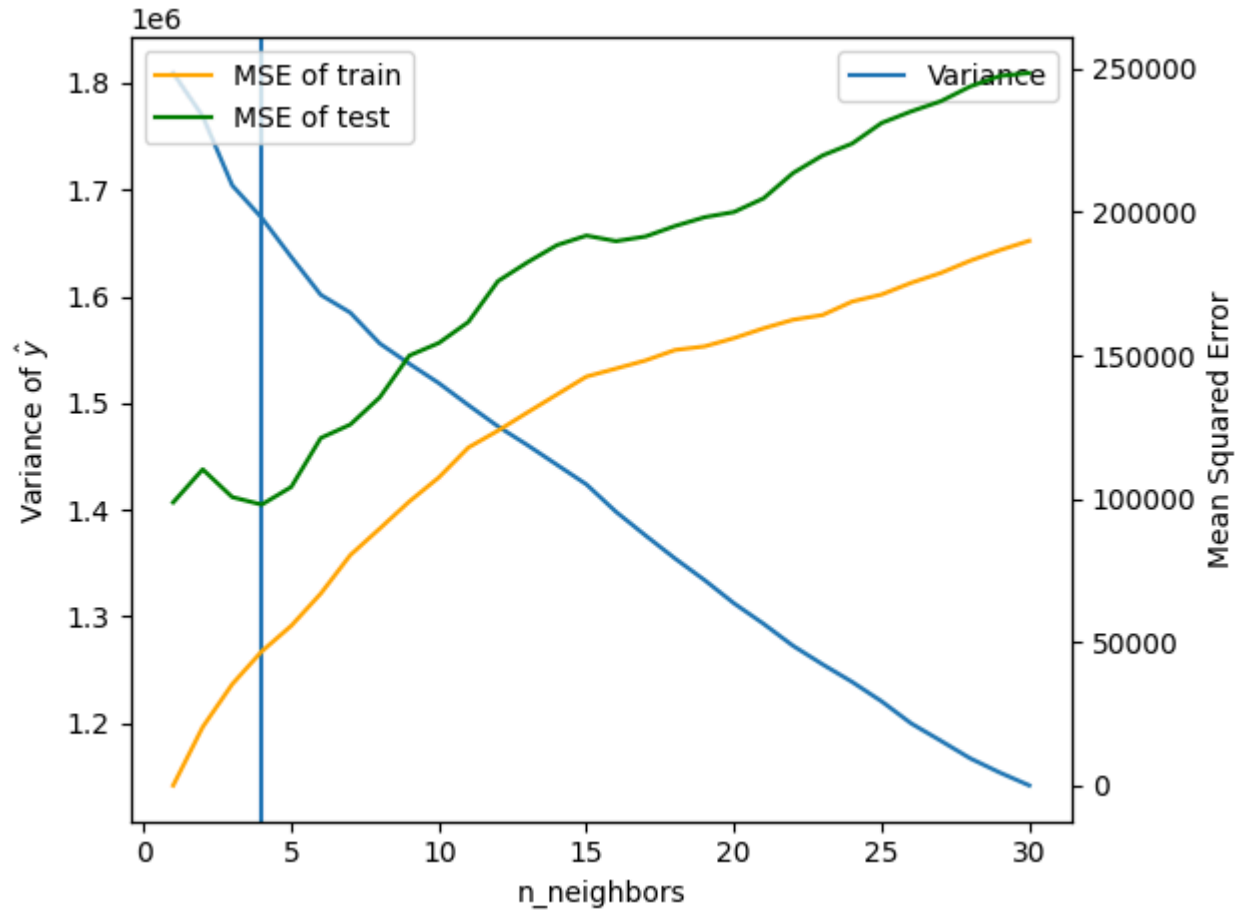
코드를 실행하면 최근접 이웃의 수에 따른 출력의 분산과
 학습 데이터 및 평가 데이터에 대한
 MSE의 변화를 보여주는 그래프가 생성됩니다.

```

'''
# 모델의 분산, 학습 데이터 성능, 평가 데이터 성능을
# 직선 그래프로 출력합니다.
plt.plot(np.arange(1, len(v) + 1), v, label='Variance')
plt.gca().set_xlabel('n_neighbors')
plt.gca().set_ylabel('Variance of  $\hat{y}$ ')
plt.legend()
plt.axvline(np.argmax(mse_test) + 1)
tax = plt.gca().twinx()
tax.plot(np.arange(1, len(v) + 1), mse_train, label = 'MSE of train', color='orange')
tax.plot(np.arange(1, len(v) + 1), mse_test, label = 'MSE of test', color='green')
tax.set_ylabel('Mean Squared Error')
plt.legend()

```

```
plt.tight_layout()
plt.show()
```



과적합의 진단

검증을 통해서 합니다.

- 학습데이터에 더 맞출수록 검증 데이터에 대한

성능이 떨어지게 되면 과적합이 진행된 것입니다.

[Ex.2]

Ex.1에서 학습 후 평가 이전에 최적합 상태를 보여주는

근접 이웃수를 검증을 통해 찾는 과정을 추가합니다.

검증 방법: 5겹 교차검증(5-fold cross validation)

In [7]:

```
'''
코드는 주어진 회귀 모델의 성능을 평가하고
검증하기 위한 함수를 정의합니다.

1. `cv = KFold(n_splits=5, random_state=123, shuffle=True)`:
   - K-Fold 교차 검증을 위한 KFold 객체를 생성합니다.
   이 경우에는 5개의 폴드로 나누고,
   랜덤 시드를 123으로 고정하고 데이터를 섞습니다.
2. `eval_model(reg)` 함수:
   - 모델의 성능을 검증하는 함수입니다.
   - 매개변수로는 회귀 모델 객체(`reg`)를 입력으로 받습니다.
   - `cross_validate()` 함수를 사용하여 교차 검증을 수행합니다.
   이 함수는 교차 검증을 통해 각 폴드에 대한
   훈련 및 검증 세트에 대한 성능을 반환합니다.
   - 교차 검증에서는 음의 평균 제곱 오차(neg_mean_squared_error)를
   사용하여 성능을 측정합니다.
   - 훈련 세트와 검증 세트에 대한 평균 및 표준 편차를
   계산하여 반환합니다.
3. `test_model(reg)` 함수:
   - 모델을 평가하는 함수입니다.
   - 매개변수로는 회귀 모델 객체(`reg`)를 입력으로 받습니다.
   - 모델을 학습시키고, 평가 데이터에 대한
   평균 제곱 오차(MSE)를 계산하여 반환합니다.

함수들은 모델을 훈련하고 평가하는데 사용될 수 있습니다.
`eval_model()`은 교차 검증을 통해 모델의 성능을 평가하고,
`test_model()`은 테스트 데이터에 대한 성능을 평가합니다.
'''
```

'''
교차 검증에서 평가 지표로 음의 평균 제곱 오차(negative mean squared error, `neg_mean_squared_error`)를 사용하는 것은 회귀 모델의 성능을 측정하는 방법 중 하나입니다. 일반적으로 회귀 모델에서는 평균 제곱 오차(MSE)를 사용하여 모델의 예측값과 실제 값 사이의 차이를 측정합니다.

그러나 `sklearn`의 ``cross_validate()`` 함수와 같은 일부 함수들은 평가 지표로 최적화할 때 값이 커지는 것을 기대하는 지표를 사용합니다. 이런 경우, 음의 MSE가 사용됩니다. 즉, 작을수록 모델의 성능이 좋다고 간주합니다. 그렇기 때문에, ``cross_validate()`` 함수에서 음의 MSE를 사용하면 작은 값일수록 모델의 성능이 우수하다고 판단할 수 있습니다.

따라서 "교차 검증에서는 음의 평균 제곱 오차를 사용하여 성능을 측정한다"는 말은 교차 검증을 수행할 때 `sklearn`의 함수들은 평가 지표로 음의 MSE를 사용한다는 것을 의미합니다.
'''

```
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_validate

cv = KFold(n_splits=5, random_state=123, shuffle=True)

def eval_model(reg):
    """
    모델의 성능을 검증합니다.
    Parameters:
        reg: Regressor
            분류기 인스턴스
    Returns:
        학습 평균, 학습 표준편차, 검증 평균, 검증 표준편차
    """
    result = cross_validate(reg, X_train, y_train, cv=cv,
                            scoring='neg_mean_squared_error',
                            return_train_score=True)
    return -np.mean(result['train_score']),
           np.std(result['train_score']),
           -np.mean(result['test_score']),
           np.std(result['test_score'])

def test_model(reg):
```

```

"""
모델을 평가합니다.
Parameters:
    reg: Regressor
        분류기 인스턴스
Returns:
    평가 결과 (Accuracy)
"""
reg.fit(X_train, y_train)
return mean_squared_error(y_test, reg.predict(X_test))

```

In [8]:

```

'''
코드는 K 최근접 이웃(K-Nearest Neighbors, KNN)
회귀 모델을 생성하고, 이를 이용하여
모델의 성능을 교차 검증을 통해 평가합니다.

1. `reg_knn = KNeighborsRegressor(n_neighbors=1)` :
    - 최근접 이웃의 수를 1로 설정하여
    KNN 회귀 모델을 생성합니다.
    이 모델은 하나의 최근접 이웃만을 고려하여 예측을 수행합니다.

2. `eval_model(reg_knn)` :
    - `eval_model()` 함수를 호출하여 모델의 성능을 검증합니다.
    - `reg_knn`으로 생성한 KNN 회귀 모델을 매개변수로 전달합니다.
    - `eval_model()` 함수는 교차 검증을 통해 모델의 성능을 평가하고,
    훈련 세트와 검증 세트에 대한 평균 및
    표준 편차를 계산하여 반환합니다.
    - 이 반환된 값은 훈련 세트에 대한 평균 및 표준 편차,
    검증 세트에 대한 평균 및 표준 편차를 포함한
    튜플 형태로 나타냅니다.

이 튜플의 요소들은 다음과 같이 해석될 수 있습니다:

- 첫 번째 요소: 훈련 세트에 대한 평균 음의 평균 제곱 오차
    (negative mean squared error, neg_mean_squared_error)
- 두 번째 요소: 훈련 세트에 대한 평균 음의 평균 제곱 오차의 표준 편차
- 세 번째 요소: 검증 세트에 대한 평균 음의 평균 제곱 오차
- 네 번째 요소: 검증 세트에 대한 평균 음의 평균 제곱 오차의 표준 편차

따라서 `eval_model(reg_knn)`을 실행하면

```

이 모델의 성능이 교차 검증을 통해
평가되고, 훈련 세트와 검증 세트에 대한
평균 MSE 및 표준 편차가 반환됩니다.
'''

```
reg_knn = KNeighborsRegressor(n_neighbors=1)
eval_model(reg_knn)
```

Out[8]: (-0.0, 0.0, 93900.00281430126, 19900.482755501595)

In [9]: '''
코드는 다양한 최근접 이웃의 수에 대해
K 최근접 이웃(K-Nearest Neighbors, KNN) 회귀 모델을 생성하고,
각각의 모델을 교차 검증을 통해 평가하여
훈련 세트와 검증 세트에 대한 평균 제곱 오차(MSE)를
계산하는 과정을 나타냅니다.

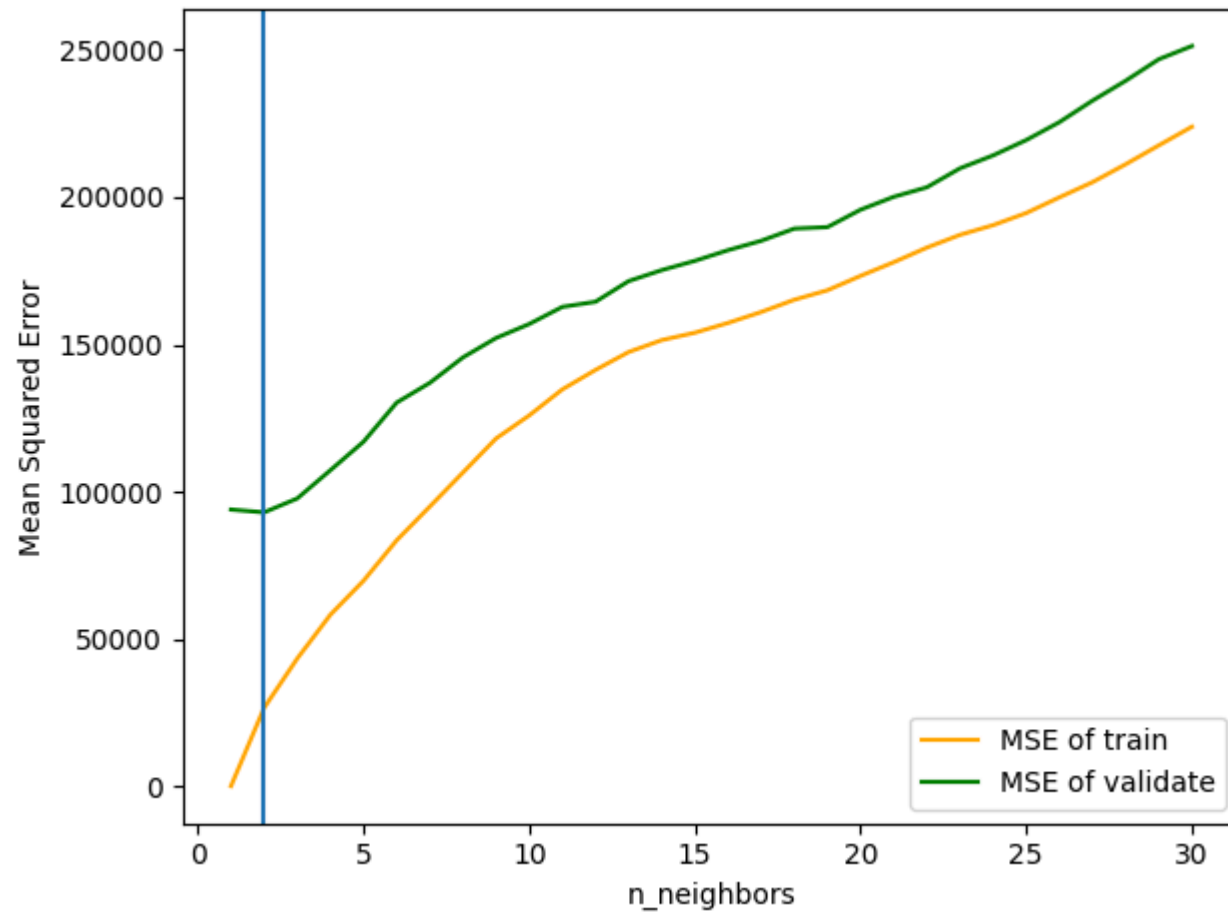
1. ``mse_train = list()``와 ``mse_validate = list()``:
- 훈련 세트와 검증 세트에 대한
MSE 값을 저장할 빈 리스트를 초기화합니다.
2. ``n_neighbors_list = np.arange(1, 31)``:
- 1부터 30까지의 범위에서 최근접 이웃의 수를 지정합니다.
3. ``for i in n_neighbors_list:``:
- ``n_neighbors_list``에 대해 반복문을 실행합니다.
4. ``reg_knn = KNeighborsRegressor(n_neighbors=i)``:
- 현재 최근접 이웃의 수(``i``)를 사용하여
KNN 회귀 모델을 생성합니다.
5. ``mse_train_, _, mse_validate_, _ = eval_model(reg_knn)``:
- 앞서 정의한 ``eval_model()`` 함수를 사용하여 생성된 모델에
대한 훈련 세트와 검증 세트에 대한 MSE 값을 계산합니다.
- 반환된 튜플에서 훈련 세트와 검증 세트에 대한
MSE 값만 사용합니다. 표준 편차는 사용하지 않습니다.
- 훈련 세트에 대한 MSE 값은 ``mse_train_``에 저장되고,
검증 세트에 대한 MSE 값은
``mse_validate_``에 저장됩니다.

6. ``mse_train.append(mse_train_)``와
``mse_validate.append(mse_validate_)`` :
 - 계산된 훈련 세트와 검증 세트에 대한
 MSE 값을 각각의 리스트에 추가합니다.

코드를 실행하면 최근접 이웃의 수가 변할 때마다
 해당 수에 대한 KNN 회귀 모델을 생성하고,
 교차 검증을 통해 훈련 세트와 검증 세트에 대한
 MSE를 계산하여 각각의 리스트에 저장합니다.
 이를 통해 최적의 최근접 이웃의 수를 찾거나
 모델의 성능을 비교할 수 있습니다.
 ...

```
mse_train = list()
mse_validate = list()
n_neighbors_list = np.arange(1, 31)
for i in n_neighbors_list:
    reg_knn = KNeighborsRegressor(n_neighbors=i)
    mse_train_, _, mse_validate_, _ = eval_model(reg_knn)
    mse_train.append(mse_train_)
    mse_validate.append(mse_validate_)
```

```
In [10]: plt.plot(n_neighbors_list, mse_train,
                  label = 'MSE of train', color='orange')
plt.plot(n_neighbors_list, mse_validate,
          label = 'MSE of validate', color='green')
plt.axvline(n_neighbors_list[np.argmin(mse_validate)])
plt.gca().set_xlabel('n_neighbors')
plt.gca().set_ylabel('Mean Squared Error')
plt.legend()
plt.tight_layout()
plt.show()
```



```
In [11]: n_neighbors_list[np.argmin(mse_test)]
```

```
Out[11]: 4
```

```
In [12]: '''  
코드는 테스트 데이터에 대해 성능을 평가하기 위해  
최적의 최근접 이웃(K-Nearest Neighbors, KNN) 회귀 모델을 생성하고,  
이 모델을 사용하여 테스트 데이터에 대한 예측을 수행합니다.
```

```
1. `reg_knn  
   = KNeighborsRegressor(n_neighbors
```

```

        = n_neighbors_list[np.argmin(mse_test)]]`:
- `np.argmin(mse_test)`를 통해 평가 데이터에 대한 MSE가
  가장 작은 값의 인덱스를 찾습니다.
  이는 평가 데이터에 대해 가장 성능이 우수한
  최근접 이웃의 수를 나타냅니다.
- `n_neighbors_list[np.argmin(mse_test)]`를 통해
  해당 인덱스에 대응하는 최적의 최근접 이웃의 수를 가져옵니다.
- 이 최적의 최근접 이웃의 수를 사용하여
  새로운 KNN 회귀 모델을 생성합니다.

2. `test_model(reg_knn)` :
- 앞서 생성한 최적의 KNN 회귀 모델(`reg_knn`)을
  매개변수로 전달하여 `test_model()` 함수를 호출합니다.
- `test_model()` 함수는 이 모델을 사용하여 테스트 데이터에 대한
  예측을 수행하고, 평균 제곱 오차(MSE)를 반환합니다.
- 반환된 MSE는 모델의 성능을 나타내며,
  더 작은 값일수록 모델의 성능이 우수합니다.

```

코드를 실행하면 평가 데이터에 대해 성능이 가장 우수한
최적의 최근접 이웃의 수를 기반으로한 KNN 회귀 모델을 생성하고,
이를 사용하여 테스트 데이터에 대한 성능을 평가합니다.
...

```

reg_knn
    = KNeighborsRegressor(n_neighbors
                          = n_neighbors_list[np.argmin(mse_test)])
test_model(reg_knn)

```

Out[12]: 97959.39558000008

모델 검증 결과 분석

- 모델의 개선 방향을 찾기 위해,

모델의 예측 오차의 양상을 살펴보고, 개선 방안을 마련합니다.

[Ex.3]

[Ex.2]의 교차 검증에서의 검증 데이터의 예측 결과의 잔차를 시각화해봅니다.

In [13]:

```
'''
코드는 교차 검증을 통해 K 최근접 이웃(K-Nearest Neighbors)
회귀 모델을 훈련하고, 이 모델을 사용하여
검증 데이터에 대한 예측 결과를 계산합니다.

1. `reg_knn = KNeighborsRegressor(n_neighbors=2)` :
    - 최근접 이웃의 수를 2로 설정하여 KNN 회귀 모델을 생성합니다.

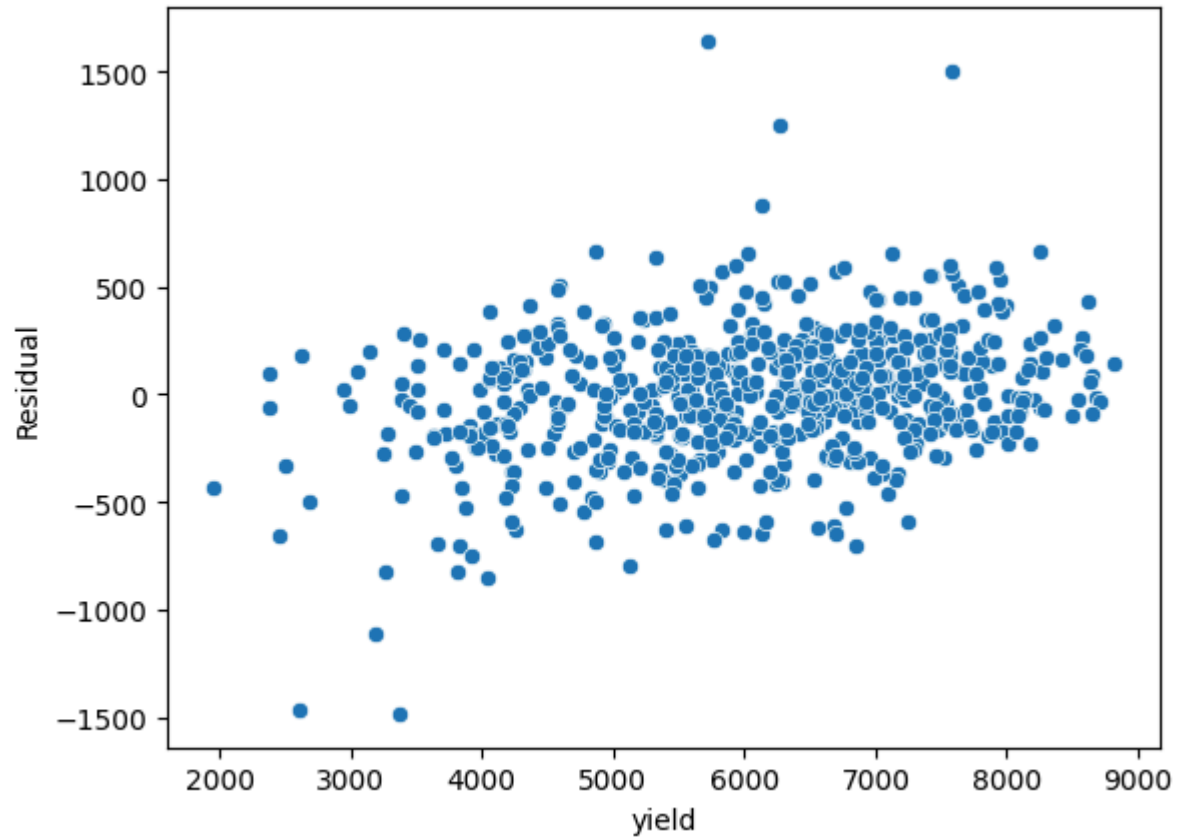
2. `y_cv = cross_val_predict(reg_knn, X_train, y_train, cv=cv)` :
    - `cross_val_predict()` 함수를 사용하여 교차검증을 통해
      훈련 데이터를 사용하여 모델을 훈련하고,
      각 폴드에 대해 검증 데이터의 예측 결과를 반환합니다.
    - 매개변수로는 생성한 KNN 회귀 모델(`reg_knn`),
      훈련 데이터(X_train, y_train),
      교차 검증 객체(`cv`)를 전달합니다.
    - 이를 통해 검증 데이터에 대한 예측 결과(`y_cv`)를 얻습니다.

3. `resid = y_train - y_cv` :
    - 훈련 데이터의 실제 값(`y_train`)에서 검증 데이터에 대한
      예측 결과(`y_cv`)를 빼서 잔차(residuals)를 계산합니다.
    - 잔차는 실제 값과 모델의 예측값 사이의 차이를 나타내며,
      모델의 성능 및 오차를 평가하는 데 사용될 수 있습니다.

코드를 실행하면 교차 검증을 통해 훈련된 KNN 회귀 모델을
사용하여 검증 데이터에 대한 예측 결과를 계산하고,
이를 통해 잔차를 얻게 됩니다.
'''

# 검증 데이터들의 예측 결과를 뽑아 줍니다.
from sklearn.model_selection import cross_val_predict
reg_knn = KNeighborsRegressor(n_neighbors=2)
y_cv = cross_val_predict(reg_knn, X_train, y_train, cv=cv)
resid = y_train - y_cv
```

```
In [14]: sns.scatterplot(x = y_train, y = resid)  
plt.gca().set_ylabel("Residual")  
plt.show()
```



[Ex.4]

fruitset, fruitmass, seed와 잔차와의 관계를 살펴봅니다.

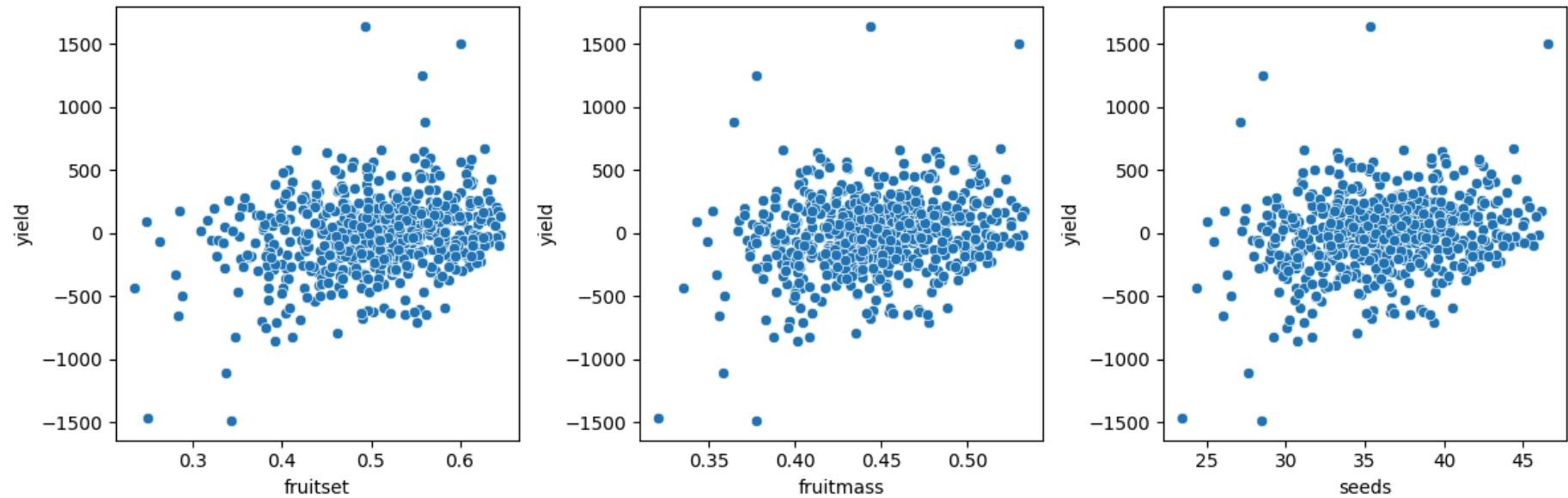
```
In [15]: '''  
코드 는 잔차(residuals)와 특성(feature) 간의 관계를  
시각화하여 모델의 성능을 평가하는 데 사용됩니다.
```

1. `fig, axes = plt.subplots(1, 3, figsize=(12, 4))`:`
 - 가로 방향으로 3개의 서브 플롯을 가진 `figure`를 생성합니다.
 - `figure` 객체는 `fig``에 할당되고, 각 서브 플롯의 축(`axis`)은 `axes`` 리스트에 할당됩니다.
 - 그림의 크기는 `(12, 4)``로 설정됩니다.
2. `for i, ax in zip(['fruitset', 'fruitmass', 'seeds'], axes):`:`
 - 각 특성(`feature`)과 잔차(`residuals`) 사이의 관계를 시각화하기 위해 반복문을 사용합니다.
 - `['fruitset', 'fruitmass', 'seeds']`` 리스트의 각 특성에 대해 반복합니다.
 - `i``에는 특성의 이름이 할당되고,
 - `ax``에는 해당 특성과 관련된 축이 할당됩니다.
3. `sns.scatterplot(x = df_train[i], y = resid, ax=ax)`:`
 - Seaborn의 `scatterplot()`` 함수를 사용하여 잔차와 각 특성(`feature`)의 관계를 산점으로 시각화합니다.
 - `x`` 축에는 특성(`feature`)의 값이,
 - `y`` 축에는 잔차(`residuals`)가 위치합니다.
 - 각 특성(`feature`)에 대한 산점도가 해당 축(`ax``)에 그려집니다.
4. `plt.tight_layout()`:`
 - 서브 플롯 간의 간격을 조정하여 레이아웃을 보다 깔끔하게 만듭니다.

코드를 실행하면 각 특성(`feature`)과 잔차(`residuals`) 간의 관계를 산점으로 시각화한 그림이 생성됩니다.
이를 통해 각 특성이 모델의 잔차에 어떤 영향을 미치는지를 시각적으로 확인할 수 있습니다.

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

for i, ax in zip(['fruitset', 'fruitmass', 'seeds'], axes):
    sns.scatterplot(x = df_train[i], y = resid, ax=ax)
plt.tight_layout()
plt.show()
```



```
In [16]: from scipy.stats import pearsonr
pearsonr(df_train['seeds'], resid)
```

```
Out[16]: (0.219095244967785, 3.468394291716062e-08)
```

[Ex.5]

KNN은 데이터의 밀도에 따라 영향도가 있습니다.

PCA로 2차원으로 축소해보고 데이터의 밀집도에 따른 성능을 살펴봅니다.

```
In [17]: '''
코드는 주성분 분석(Principal Component Analysis, PCA)을
사용하여 데이터를 2차원으로 축소하고,
이를 시각화하여 데이터의 구조를 살펴봅니다.

1. `from sklearn.decomposition import PCA`:
   - scikit-learn의 PCA 모듈을 임포트합니다.
2. `pca = PCA(n_components=2)`:
   - PCA 객체를 생성합니다. 여기서는 주성분을 2개로
   설정하여 2차원으로 데이터를 축소합니다.
```

3. `pca_val = pca.fit_transform(df_train)`:
 - 주어진 훈련 데이터(`df_train`)를 PCA 모델에 적합시키고, 데이터를 주성분 축으로 변환합니다.
 - `fit_transform()` 메서드를 사용하여 주성분을 추출하고, 데이터를 새로운 주성분 축으로 변환합니다.
 - 결과로 얻은 `pca_val`은 2차원 공간상의 주성분 축을 기반으로한 데이터의 표현입니다.
4. `sns.scatterplot(x=pca_val[:, 0], y = pca_val[:, 1], c=resid)`:
 - Seaborn의 `scatterplot()` 함수를 사용하여 PCA로 변환된 데이터를 2차원 산점도로 시각화합니다.
 - x 축에는 첫 번째 주성분(`pca_val[:, 0]`), y 축에는 두 번째 주성분(`pca_val[:, 1]`)을 사용합니다.
 - `c=resid`를 통해 잔차(residuals)를 색상으로 표시합니다. 잔차가 클수록 더 진한 색으로 표시됩니다.
5. `plt.xlabel('x1')`와 `plt.ylabel('x2')`:
 - x 축과 y 축에 레이블을 추가합니다. 여기서 'x1'과 'x2'는 PCA로 얻은 주성분 축을 나타냅니다.

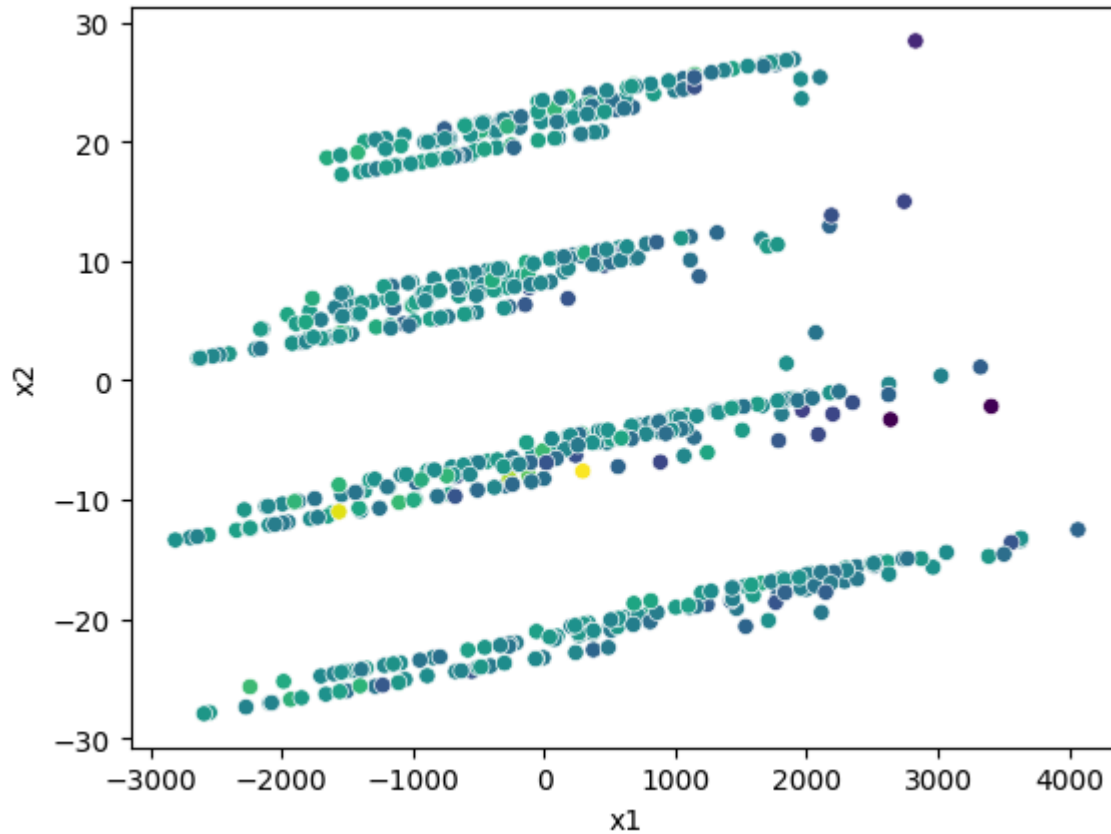
코드를 실행하면 PCA를 사용하여 데이터를 2차원으로 축소한 후, 그 결과를 시각화한 2차원 산점도가 생성됩니다. 각 점은 데이터의 주성분에 대응하며, 잔차의 크기에 따라 색상이 지정됩니다. 이를 통해 데이터의 패턴과 잔차 간의 관계를 시각적으로 확인할 수 있습니다.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_val = pca.fit_transform(df_train)

sns.scatterplot(x=pca_val[:, 0], y = pca_val[:, 1], c=resid)
plt.xlabel('x1')
plt.ylabel('x2')

plt.show()
```



3. 최적 모델 선택

하이퍼 파라미터 최적화(Hyper Parameter Optimization)

최적의 성능을 보이는 머신러닝 모델의 설정을 탐색합니다.

→ 모델이 나타내는 데이터의 형태. Ex) SVM의 Kernel 함수, 인공신경망의 신경망 구성, ...

→ 모델의 복잡도를 탐색. Ex) 의사결정나무의 나무의 최대깊이, 리프노드 수, ...

→ 모델의 파라미터(계수, 가중치, ...)의 절대 크기에 대한 규제(Regularization) 계수

→ ...

Grid Search

여러 하이퍼 파라미터간의 최적의 조합을 찾아내는 방법입니다.

하이퍼 파라미터 별로 탐색하고자 하는 값들을 설정하고,

여러 하이퍼 파라미터에서 조합할 수 있는 경우에 대해서 성능을 측정합니다.

```
In [6]: Image('126.png')
```

```
Out[6]: sklearn.model_selection.GridSearchCV
```

주요 하이퍼파라미터

이름	설명
estimators	GridSearch 대상 모델
param_grid	탐색할 하이퍼파라미터 셋 키(Key)는 하이퍼파라미터명 값은 후보 리스트 인 dictionary구조
scoring	선택의 기준이 되는 성능 지표
cv	검증 방법을 나타내는 Splitter

[Ex.6]

df_train에서 다음 Random Forest Regressor

하이퍼 파라미터 설정에서 최적의 성능을 보이는 조합을 찾아 봅니다.

```
n_estimators: [30, 50, 70]
```

```
max_depth: [7, 8, 9]
```

```
min_samples_leaf: [4, 8]
```

검증 방법: 5겹 교차검증(5 fold cross validation)

성능 지표: 평균절대오차(MAE, Mean Absolute Error)

```
In [18]: from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV, KFold

grcv = GridSearchCV(
    # 대상 모델의 인스턴스를 생성하여 전달합니다.
    estimator = RandomForestRegressor(random_state=123),
    param_grid = {
        'n_estimators': [50, 70, 100],
        'max_depth': [9, 10, 11],
        'min_samples_leaf': [2, 4]
    }, # 대상 하이퍼파라미터를 정의 합니다.
    #key: 하이퍼파라미터명, values: 탐색 후보 리스트

    # 성능 평가 기준입니다.
    scoring='neg_mean_absolute_error',

    # 검증 방법 5겹 교차 검증에 따른 Splitter를 전달합니다.
    cv = KFold(n_splits=5, random_state=123, shuffle=True)
)
grcv.fit(X_train, y_train)
```

```
Out[18]: GridSearchCV(cv=KFold(n_splits=5, random_state=123, shuffle=True),
                      error_score='raise-deprecating',
                      estimator=RandomForestRegressor(bootstrap=True, criterion='mse',
                                                         max_depth=None,
                                                         max_features='auto',
                                                         max_leaf_nodes=None,
                                                         min_impurity_decrease=0.0,
                                                         min_impurity_split=None,
                                                         min_samples_leaf=1,
                                                         min_samples_split=2,
                                                         min_weight_fraction_leaf=0.0,
                                                         n_estimators='warn', n_jobs=None,
                                                         oob_score=False, random_state=123,
                                                         verbose=0, warm_start=False),
                      iid='warn', n_jobs=None,
                      param_grid={'max_depth': [9, 10, 11], 'min_samples_leaf': [2, 4],
                                   'n_estimators': [50, 70, 100]},
                      pre_dispatch='2*n_jobs', refit=True, return_train_score=False,
                      scoring='neg_mean_absolute_error', verbose=0)
```

```
In [19]: grcv.best_params_, grcv.best_score_
```

```
Out[19]: ({'max_depth': 11, 'min_samples_leaf': 2, 'n_estimators': 70},
          -118.68269012690861)
```

```
In [20]: grcv.cv_results_
```

```

Out[20]: {'mean_fit_time': array([0.0718195 , 0.10296702, 0.14860516, 0.06357107, 0.08879352,
    0.12634087, 0.07300029, 0.10392857, 0.1514051 , 0.06321235,
    0.09129663, 0.13280478, 0.07405629, 0.10180616, 0.14520168,
    0.06420097, 0.08879485, 0.13075857]),
  'std_fit_time': array([0.0003962 , 0.00062316, 0.00515442, 0.00044505, 0.00041305,
    0.00138591, 0.00107966, 0.001623 , 0.00233408, 0.00040838,
    0.00260227, 0.00256306, 0.00096078, 0.00075057, 0.00129911,
    0.00077424, 0.00038226, 0.00148478]),
  'mean_score_time': array([0.00261788, 0.0036056 , 0.00500064, 0.00280089, 0.00340629,
    0.00480018, 0.00279989, 0.00380549, 0.00499401, 0.00279312,
    0.0037941 , 0.00539479, 0.00299907, 0.00359445, 0.00499063,
    0.00239887, 0.0040051 , 0.00509896]),
  'std_score_time': array([5.07704589e-04, 4.94326643e-04, 2.00838467e-06, 4.00522021e-04,
    4.96582003e-04, 3.97849865e-04, 4.00019843e-04, 4.03006298e-04,
    1.17602003e-05, 3.95513684e-04, 7.55002951e-04, 4.82808551e-04,
    1.53775363e-06, 4.83995386e-04, 1.88645603e-05, 4.90745704e-04,
    1.20448349e-05, 1.51404290e-04]),
  'param_max_depth': masked_array(data=[9, 9, 9, 9, 9, 9, 10, 10, 10, 10, 10, 10, 11, 11, 11,
    11, 11, 11],
    mask=[False, False, False, False, False, False, False, False, False,
    False, False, False, False, False, False, False, False],
    fill_value='?',
    dtype=object),
  'param_min_samples_leaf': masked_array(data=[2, 2, 2, 4, 4, 4, 2, 2, 2, 4, 4, 4, 2, 2, 2, 4, 4, 4],
    mask=[False, False, False, False, False, False, False, False, False,
    False, False, False, False, False, False, False, False],
    fill_value='?',
    dtype=object),
  'param_n_estimators': masked_array(data=[50, 70, 100, 50, 70, 100, 50, 70, 100, 50, 70, 100, 50,
    70, 100, 50, 70, 100],
    mask=[False, False, False, False, False, False, False, False, False,
    False, False, False, False, False, False, False, False],
    fill_value='?',
    dtype=object),
  'params': [{'max_depth': 9, 'min_samples_leaf': 2, 'n_estimators': 50},
    {'max_depth': 9, 'min_samples_leaf': 2, 'n_estimators': 70},
    {'max_depth': 9, 'min_samples_leaf': 2, 'n_estimators': 100}],

```

```

{'max_depth': 9, 'min_samples_leaf': 4, 'n_estimators': 50},
{'max_depth': 9, 'min_samples_leaf': 4, 'n_estimators': 70},
{'max_depth': 9, 'min_samples_leaf': 4, 'n_estimators': 100},
{'max_depth': 10, 'min_samples_leaf': 2, 'n_estimators': 50},
{'max_depth': 10, 'min_samples_leaf': 2, 'n_estimators': 70},
{'max_depth': 10, 'min_samples_leaf': 2, 'n_estimators': 100},
{'max_depth': 10, 'min_samples_leaf': 4, 'n_estimators': 50},
{'max_depth': 10, 'min_samples_leaf': 4, 'n_estimators': 70},
{'max_depth': 10, 'min_samples_leaf': 4, 'n_estimators': 100},
{'max_depth': 11, 'min_samples_leaf': 2, 'n_estimators': 50},
{'max_depth': 11, 'min_samples_leaf': 2, 'n_estimators': 70},
{'max_depth': 11, 'min_samples_leaf': 2, 'n_estimators': 100},
{'max_depth': 11, 'min_samples_leaf': 4, 'n_estimators': 50},
{'max_depth': 11, 'min_samples_leaf': 4, 'n_estimators': 70},
{'max_depth': 11, 'min_samples_leaf': 4, 'n_estimators': 100}],
'split0_test_score': array([-125.48956048, -124.66804962, -124.89408768, -131.10004033,
-130.31865294, -131.46283361, -125.5200655 , -124.67441587,
-124.70159083, -130.97345742, -130.18513313, -131.34402703,
-125.14427561, -124.18306994, -124.15195535, -130.97345742,
-130.18513313, -131.34402703]),
'split1_test_score': array([-134.54407418, -135.96769541, -137.00250851, -145.50251269,
-145.85856817, -146.13719021, -135.25811932, -136.3076772 ,
-137.09357329, -145.77650326, -145.96471386, -146.22622168,
-135.19135744, -135.94115806, -137.02757587, -145.77650326,
-145.91732588, -146.1930501 ]),
'split2_test_score': array([-107.47863241, -107.07770587, -106.87016929, -118.81988623,
-119.88845217, -119.68256275, -107.16502658, -106.64326475,
-106.66058176, -118.759208 , -119.91772487, -119.70479131,
-107.3411479 , -106.71747386, -106.95984662, -118.759208 ,
-119.91772487, -119.70479131]),
'split3_test_score': array([-110.94402358, -110.49784434, -111.97394059, -122.78690902,
-122.42221922, -124.14977502, -110.48569028, -110.19937139,
-111.84418657, -122.82297536, -122.46265218, -124.17916452,
-110.97901167, -110.94897637, -112.4553542 , -122.82297536,
-122.46265218, -124.17916452]),
'split4_test_score': array([-116.01148518, -115.50127006, -114.97744341, -120.61198577,
-120.5468062 , -119.41540328, -117.78778025, -116.50731302,
-115.25673753, -120.60589723, -120.55190024, -119.36641277,
-116.1357658 , -115.5784145 , -114.44446898, -120.60589723,
-120.55190024, -119.36641277]),
'mean_test_score': array([-118.90417675, -118.75205499, -119.15288989, -127.76963842,

```

```

-127.81098437, -128.17485616, -119.25344384, -118.87576112,
-119.12033602, -127.79273844, -127.8202392 , -128.16924408,
-118.96827298, -118.68269013, -119.0161238 , -127.79273844,
-127.81077687, -128.16262045]),
'std_test_score': array([ 9.89755687, 10.44767216, 10.68412259,  9.80834   ,  9.75561858,
  9.97786429, 10.19852535, 10.65246817, 10.73893903,  9.90733756,
  9.77825063, 10.00465195, 10.07830722, 10.39321908, 10.57730759,
  9.90733756,  9.76069496,  9.99269888]),
'rank_test_score': array([ 4,  2,  8, 10, 14, 18,  9,  3,  7, 11, 15, 17,  5,  1,  6, 11, 13,
 16]))}

```

4. 속성 부분집합 선택(Feature Subset Selection)

In [7]: `Image('127.png')`

Out[7]: 필터(Filter) 방법

입력 변수(속성)와 대상 변수와의 연관성을 측정하고 높은 연관성을 갖는 변수를 우선으로 선택합니다.

대표적인 연관성 지표로 상관도, 정보 이득, 카이제곱 통계량 등이 있습니다.

sklearn.model_selection.SelectKBest

주요 하이퍼파라미터

이름	설명
score_func	입력 변수와 대상 변수의 관련성을 측정해주는 함수 def func_name(X, y): """ Parameters: X: input y: target Returns scores, pvalues pvalues 는 생략 가능합니다. """

k 선택할 속성(입력 변수)의 개수

[Ex.7]

[Ex 1]에서 제작한 입력 변수를 정규화 처리한 데이터인 X_train과

대상 데이터인 y_train의 피어슨 상관도를 기준으로

상관도가 높은 입력 변수(속성)를 우선으로

5개에서 16개까지 선택하여 성능을 측정했을 때,

가장 좋은 성능을 보일 때의 선택한 입력 변수의 수를 구해 봅니다.

max_depth: 11, min_samples_leaf: 2, n_estimators: 70 의

Random Forest Regressor를 사용합니다

검증 방법: 5겹 교차 검증(5-fold cross validation)

성능 측정 지표: 평균절대오차(MAE, Mean Absolute Error)

In [21]:

```
'''
코드는 다음과 같은 작업을 수행합니다:

1. `get_corr` 함수는 입력 변수 `X`와 대상 변수 `y`
   간의 피어슨 상관 관계와 p-value를 계산합니다.
2. `get_corr` 함수는 각 입력 변수와 대상 변수 간의
   피어슨 상관 관계의 절댓값과 p-value를 반환합니다.
3. `KFold`를 사용하여 데이터를 5개의 폴드로 나눕니다.
   이것은 교차 검증을 위한 데이터 분할 방법입니다.
4. `cv_scores`는 교차 검증을 통해
   각 모델의 성능을 저장할 리스트입니다.
5. `for` 루프를 통해 `SelectKBest`를 사용하여
   변수 선택 후 `RandomForestRegressor`를 구축합니다.
6. `cross_val_score` 함수를 사용하여 교차 검증을 수행하고
   평균 절대 오차의 음수 값을 계산합니다.
   이는 모델의 성능 측정입니다.
7. 각 모델의 성능을 `cv_scores` 리스트에 저장합니다.

코드는 변수 선택을 통해 RandomForestRegressor 모델을
최적화하고, 교차 검증을 통해 모델의 성능을 측정하여
최적의 변수 수를 결정하는 데 사용됩니다.
'''

from sklearn.ensemble import RandomForestRegressor
from sklearn.feature_selection import SelectKBest
from sklearn.pipeline import make_pipeline
from scipy.stats import pearsonr
from sklearn.model_selection import cross_val_score

def get_corr(X, y):
```

```

"""
입력 변수 x와, 대상 변수 y 간 피어슨 상관도와
pvalue를 반환합니다.
Parameters:
    x: 입력 변수
    y: 대상 변수
Returns:
    scores(pearson correlation), pvalues
"""
scores, pvalues = list(), list()
for i in range(X.shape[1]):
    r = pearsonr(X[:, i], y)
    scores.append(abs(r[0]))
    pvalues.append(r[1])
return scores, pvalues

cv = KFold(n_splits=5, random_state=123, shuffle=True)

cv_scores = list()

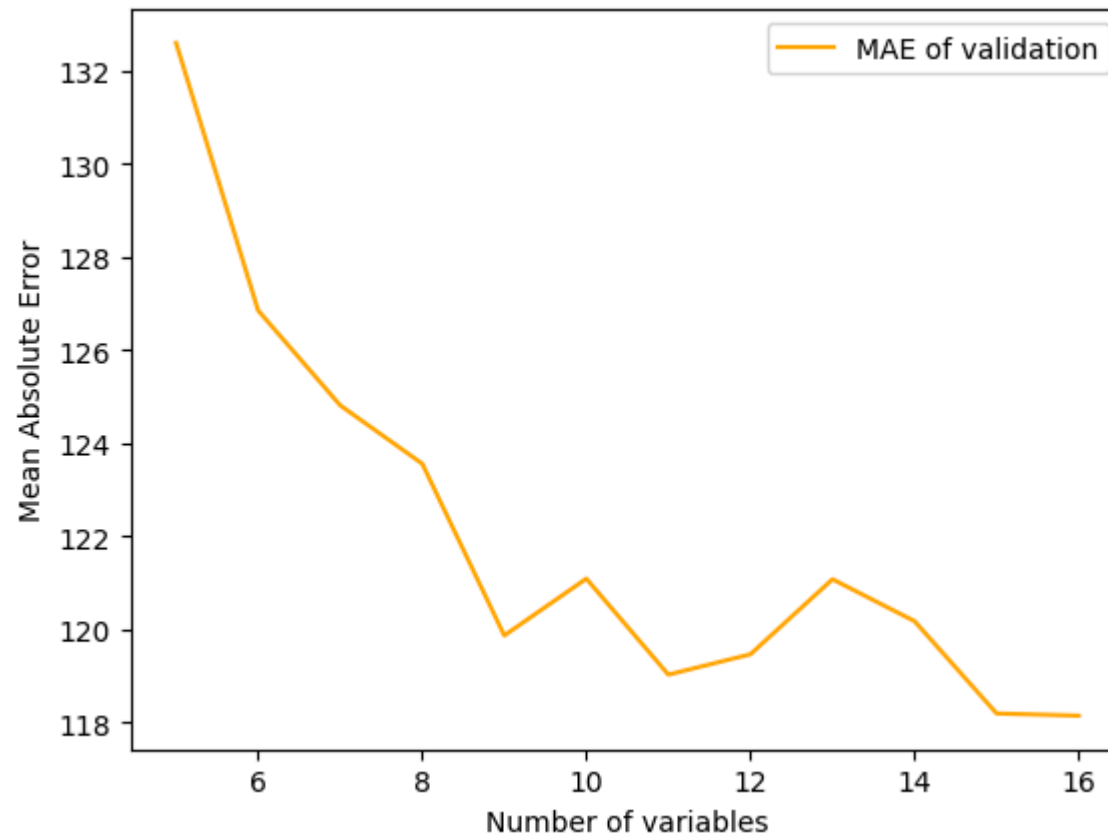
for i in range(5, 17):
    reg_rf_kbest = make_pipeline(
        SelectKBest(get_corr, k=i),
        RandomForestRegressor(max_depth=11,
                               min_samples_leaf=2,
                               n_estimators=70)
    )
    cv_scores.append(
        -np.mean(cross_val_score(reg_rf_kbest, X_train, y_train,
                                   cv=cv,
                                   scoring='neg_mean_absolute_error'))
    )

```

```

In [22]: plt.plot(np.arange(5, 17), cv_scores,
                  label = 'MAE of validation', color='orange')
plt.gca().set_xlabel('Number of variables')
plt.gca().set_ylabel('Mean Absolute Error')
plt.legend()
plt.show()

```



WRAPPER(Wrapper)방법

사용할 모델에 선택한 변수로 검증을 하여

가장 좋은 성능을 내는 변수 조합을 찾습니다.

예시를 위해 입력 변수의 수가 적은 sklearn에 내장된

Iris 데이터셋을 불러 옵니다.

80%는 학습데이터($X_{\text{iris_train}}$, $y_{\text{iris_train}}$)로,

20%(X_iris_test, y_iris_test)로 나누어 봅니다.

이 때, 계층적 선택법을 사용합니다.

```
In [8]: from sklearn.datasets import load_iris

iris = load_iris()
X_iris = iris['data']
y_iris = iris['target']

X_iris_train, X_iris_test, y_iris_train, y_iris_test \
    = train_test_split(X_iris, y_iris,
                      train_size=0.8, stratify=y_iris)
```

```
In [24]: iris['feature_names']
```

```
Out[24]: ['sepal length (cm)',
          'sepal width (cm)',
          'petal length (cm)',
          'petal width (cm)']
```

```
In [9]: Image('128.png')
```

Out[9]: **All Subsets**

입력 변수에서 조합할 수 있는 모든 변수의 조합을 사용합니다.

※ 시간 복잡도가 $O(2^n)$ (n은 입력 변수의 수)이므로

입력 변수가 많아 질수록, 수행 속도가 급격하게 증가합니다.

[Ex.8]

학습 데이터 {X_iris_train, y_iris_train} 에서

최적의 입력 변수 조합을 All Subsets 방법으로 찾아 봅니다.

성능 지표는 정확도(Accuracy)입니다.

Wrapper는 로지스틱 회귀모델을 사용합니다.

찾은 변수 조합으로 평가 데이터

{X_iris_test, Y_iris_test}의 성능을 측정 합니다.

In [15]:

```
'''
1. `StratifiedKFold`를 사용하여 데이터를 계층화된
   5개의 폴드로 나눕니다. 이는 계층화된 교차 검증을
   위한 데이터 분할 방법입니다.
2. 빈 리스트인 `scores`와 `subsets`를 초기화합니다.
   `scores`는 각 변수 부분 집합에 대한
   교차 검증 점수를 저장하고,
   `subsets`는 해당 변수 부분 집합을 저장합니다.
3. Logistic 회귀 모델(`LogisticRegression`)을 구축합니다.
   이 모델은 StandardScaler로 전처리된 입력 데이터를
   사용하여 학습됩니다.

`make_pipeline` 함수는 여러 개의 변환기와 추정기를 연결하여
하나의 파이프라인을 만드는 유틸리티 함수입니다.
이 함수를 사용하면 간단하고 직관적으로
파이프라인을 구축할 수 있습니다.
`StandardScaler`는 입력 데이터를 평균을 0으로,
표준 편차를 1로 변환하는 전처리 스텝입니다.
이는 데이터의 스케일을 조정하여 학습 과정을 안정화하고
성능을 향상시키는 데 도움이 됩니다.
`LogisticRegression`은 로지스틱 회귀 모델을
구축하는 함수입니다. 이 모델은 주어진 입력에 대해
다중 클래스 분류(multi-class classification)를 수행할 수 있습니다.
`solver='lbfgs'`는 최적화에 사용되는 알고리즘을 지정합니다.
'lbfgs'는 한 유형의 최적화 알고리즘이며,
주로 작은 규모의 데이터셋에서 사용됩니다.
`multi_class='multinomial'`은 다중 클래스 분류를
지원하는 손실 함수를 선택하는 인수입니다.
따라서 `clf_lr` 파이프라인은 다음과 같은 순서로
데이터를 처리하고 모델을 학습합니다:
```

1. ``StandardScaler``를 사용하여 데이터를 전처리합니다.
 2. 전처리된 데이터를 사용하여
 ``LogisticRegression`` 모델을 학습합니다.
 이렇게 구성된 파이프라인은 전처리와 모델 학습을
 하나의 단계로 묶어주므로 코드를 간결하게 유지하고
 유지보수를 용이하게 합니다.

4. ``for`` 루프를 사용하여 선택할 변수의 수를
 1부터 4까지 반복합니다.
5. ``combinations`` 함수를 사용하여 선택된 변수의
 모든 부분 집합을 생성합니다.
 ``combinations`` 함수는 지정된 범위에서
 주어진 길이의 가능한 모든 조합을 반환합니다.
6. 각 변수 부분 집합에 대해 교차 검증을 수행하고,
 정확도 점수를 ``scores`` 리스트에 저장합니다.
 이 때, 해당 부분 집합의 변수만 사용하여
 모델을 학습하고 평가합니다.
7. 모든 변수 부분 집합에 대한 교차 검증이 완료되면,
 각 부분 집합에 대한 평균 정확도 점수가
 ``scores`` 리스트에 저장됩니다.

코드는 Logistic 회귀 모델을 사용하여 변수 선택의
 각 부분 집합에 대해 교차 검증을 수행하고,
 각 부분 집합의 정확도를 평가하여
 최적의 변수 조합을 결정하는 데 사용됩니다.
 ...

```
from itertools import combinations
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score

cv = StratifiedKFold(n_splits=5,
                     random_state=123, shuffle=True)

scores = list()
subsets = list()

clf_lr = make_pipeline(StandardScaler(),
```

```

        LogisticRegression(solver='lbfgs',
                           multi_class='multinomial'))

for i in range(1, 5):
    for j in combinations(np.arange(0, 4), i):
        X_cols = j
        subsets.append(j)
        scores_ = cross_val_score(
            clf_lr,
            X_iris_train[:, j],
            y_iris_train,
            scoring = 'accuracy',
            cv = cv
        )
        scores.append(np.mean(scores_))

```

In [26]:

```

'''
1. `scores` 리스트에서 가장 높은 값을 가진
   인덱스(`best_idx`)를 찾습니다.
   이는 가장 높은 교차 검증 정확도를 가진
   변수 부분 집합을 찾는 것입니다.
2. `subsets` 리스트에서 `best_idx`에 해당하는
   부분 집합(`best_features`)을 가져옵니다.
   이는 가장 높은 교차 검증 정확도를 가진
   변수 부분 집합을 나타냅니다.
3. 선택된 변수 부분 집합의 교차 검증
   정확도(`scores[best_idx]`)와 해당 변수들의
   인덱스(`best_features`)를 출력합니다.
   이 때, `iris['feature_names']`에서
   변수 이름을 가져와서 출력합니다.
- `scores[best_idx]`:
  `scores` 리스트에서 `best_idx`에 해당하는
  값을 가져옵니다. 이는 가장 높은
  교차 검증 정확도를 나타냅니다.
- `best_features`: `subsets` 리스트에서
  `best_idx`에 해당하는 인덱스의 값을 가져옵니다.
  이는 가장 높은 교차 검증 정확도를 가진
  변수 부분 집합을 나타냅니다.
- `[iris['feature_names'][i] for i in best_features]`:
  리스트 컴프리헨션(list comprehension) 구문입니다.
'''

```

`best\_features`에 있는 각 변수의 인덱스(`i`)를 가져와서 `iris['feature\_names']`에서 해당 변수 이름을 가져와 리스트에 저장합니다.
 결과적으로는 최적의 변수들에 해당하는 변수 이름의 리스트가 생성됩니다.
 코드는 리스트의 요소들을 가져와서 새로운 리스트를 생성하는 방법을 사용하여, 최적의 변수들에 해당하는 변수 이름을 가져와 리스트로 반환합니다.

코드는 교차 검증을 통해 얻은 정확도가 가장 높은 변수 부분 집합을 찾고, 해당 부분 집합에 대한 교차 검증 정확도와 변수 이름을 출력합니다.
 이를 통해 최적의 변수 조합을 확인할 수 있습니다.
 '''

```
best_idx = np.argmax(scores)
best_features = subsets[best_idx]
scores[best_idx], best_features,
    [iris['feature_names'][i] for i in best_features]
```

Out[26]: (0.9583333333333334, (3,), ['petal width (cm)'])

In [27]: '''

1. `clf\_lr.fit(X\_iris\_train[:, best\_features], y\_iris\_train)`:
 `clf\_lr` 파이프라인(StandardScaler와 LogisticRegression으로 구성된)을 사용하여 최적의 변수들에 대해 로지스틱 회귀 모델을 학습합니다.
 이 때, 학습 데이터인 `X\_iris\_train`에서 최적의 변수들에 해당하는 열(`best\_features`)만 선택하여 사용합니다.
 대응하는 타겟 변수는 `y\_iris\_train`입니다.
2. `clf\_lr.predict(X\_iris\_test[:, best\_features])`:
 학습된 모델을 사용하여 테스트 데이터(`X\_iris\_test`)에서 예측을 수행합니다. 마찬가지로 최적의 변수들에 해당하는 열(`best\_features`)만 선택하여 사용합니다.
3. `accuracy\_score(y\_iris\_test, clf\_lr.predict(X\_iris\_test[:, best\_features]))`:
 예측된 결과와 실제 테스트 데이터의 타겟 변수(`y\_iris\_test`)를 비교하여 정확도를 계산합니다. 이를 위해 `accuracy\_score` 함수를 사용합니다.
 이 함수는 예측된 결과와 실제 결과 간의

정확도를 계산하여 반환합니다.

코드는 최적의 변수들을 사용하여 학습된 로지스틱 회귀 모델을
테스트 데이터에 적용하고, 그 결과의 정확도를 계산합니다.
이를 통해 모델이 테스트 데이터에서
얼마나 잘 수행되는지를 확인할 수 있습니다.
...

```
clf_lr.fit(X_iris_train[:, best_features],
           y_iris_train)
accuracy_score(y_iris_test,
               clf_lr.predict(X_iris_test[:, best_features]))
```

Out[27]: 0.9666666666666667

전진선택법(Forward Selection)과 후진제거법(Backward Elimination)

- 연산 정의

1. 전진 선택: 선택하지 않은 속성들을 하나 씩 추가 해봤을 때

가장 좋은 성능을 보인 속성을 선택합니다.

2. 후진 제거: 선택한 속성들을 하나 씩 제거해 봤을 때

가장 좋은 성능을 보인 속성을 제거합니다.

- 선택법

1. 전진선택법(Forward Selection)

- 빈리스트에서 시작하여, 단계적으로 전진 선택을 합니다.
- 더이상 성능에 개선이 없거나 선택할 속성이 없다면 멈춥니다.

2. 단계적 전진선택법(Stepwise Forward Selection)

- 빈리스트에서 시작하여, 단계적으로 전진 선택을 하고,
후진 제거를 하여 성능에 개선이 있다면 제거를 합니다.
- 더이상 성능에 개선이 없거나 선택할 속성이 없다면 멈춥니다.

3. 후진제거법(Backward Elimination)

- 모든 속성 리스트에서 시작하여, 단계적으로 후진 제거를 합니다.
- 더이상 성능에 개선이 없거나 제거할 속성이 없다면 멈춥니다.

4. 단계적 후진제거법(Stepwise Backward Elimination)

- 모든 속성 리스트에서 시작하여, 단계적으로 후진 제거를 하고,
전진 선택을 하여 성능에 개선이 있다면 추가 합니다.
- 더이상 성능에 개선이 없거나 제거할 속성이 없다면 멈춥니다.

[Ex.9]

[Ex.8]에서 All Subsets 예제에서 All Subsets 대신 전진선택법,
단계적 전진선택법, 후진제거법, 단계적 후진제거법을 수행해봅니다.

In [28]:

```
'''  
코드는 변수 선택을 위한 전진 선택 연산과
```

후진 제거 연산을 구현하는 함수들을 포함하고 있습니다.

1. ``eval_subsets(subsets)``:
주어진 변수 부분 집합에 대한 성능을 평가하는 함수입니다.
주어진 변수 부분 집합을 사용하여 교차 검증을 수행하고,
교차 검증 정확도의 평균을 반환합니다.
 - ``cross_val_score`` 함수를 사용하여 ``clf_lr``
파이프라인을 이용해 교차검증을 수행합니다.
 - 각 교차 검증에서 얻은 정확도를 리스트에 저장하고,
이를 평균하여 최종 정확도를 계산합니다.
 - 최종 정확도를 반환합니다.
2. ``forward_select(X_all, X_subsets)``:
전진 선택 연산을 수행하는 함수입니다.
모든 변수 집합(``X_all``)과 현재 선택된 변수 집합(``X_subsets``)을
사용하여 가능한 변수를 추가하여 성능을 평가하고,
가장 높은 성능을 갖는 변수 집합과 해당 성능을 반환합니다.
 - 현재 선택된 변수 집합(``X_subsets``)에 없는 모든 변수를
추가하여 변수 집합 후보(``X_fwd``)를 생성합니다.
 - 각 후보 변수 집합에 대해 ``eval_subsets`` 함수를
호출하여 성능을 평가합니다.
 - 가장 높은 성능을 갖는 변수 집합과 해당 성능을 반환합니다.
3. ``backward_eliminate(X_subsets)``:
후진 제거 연산을 수행하는 함수입니다.
현재 선택된 변수 집합(``X_subsets``)에서 한 변수씩 제거하여
성능을 평가하고, 가장 높은 성능을 갖는 변수 집합과
해당 성능을 반환합니다.
 - 현재 선택된 변수 집합(``X_subsets``)에서 한 변수씩
제거하여 변수 집합 후보(``X_bwd``)를 생성합니다.
 - 각 후보 변수 집합에 대해 ``eval_subsets`` 함수를
호출하여 성능을 평가합니다.
 - 가장 높은 성능을 갖는 변수 집합과 해당 성능을 반환합니다.

이러한 함수들은 변수 선택 알고리즘의 한 부분으로 사용될 수 있습니다.

전진 선택 연산과 후진 제거 연산을 반복하면서

최적의 변수 집합을 결정할 수 있습니다.

이를 통해 모델의 복잡성을 줄이고,

성능을 최대화하는 최적의 변수 집합을 찾을 수 있습니다.

...

```
from itertools import combinations
```

```

clf_lr = make_pipeline(StandardScaler(),
                       LogisticRegression(solver='lbfgs',
                                           multi_class='multinomial'))

def eval_subsets(subsets):
    """
    변수셋의 성능을 평가합니다.
    Parameters:
        subsets: list
            입력 변수 리스트
    Returns
        accuracy
    """
    return np.mean(
        cross_val_score(clf_lr, X_iris_train[:, subsets],
                        y_iris_train,
                        scoring='accuracy', cv=cv)
    )

def forward_select(X_all, X_subsets):
    """
    전진 선택 연산
    Parameters:
        X_all: list
            모든 입력 변수 리스트
        X_subsets: list
            선택된 입력 변수 리스트
    Returns
        str, float
            선택된 변수명, 성능
    """
    X_fwd = [X_subsets + [i] \
              for i in X_all if i not in X_subsets]

    scores_fwd = list()

    for i in X_fwd:
        scores_fwd.append(eval_subsets(i))
    idx = np.argmax(scores_fwd)

    return X_fwd[idx], scores_fwd[idx]

```

```
def backward_eliminate(X_subsets):
    """
    후진 제거 연산
    Parameters:
        X_subsets: list
            선택된 입력 변수 리스트
    Returns
        str, float
            선택된 변수명, 성능
    """
    if len(X_subsets) < 2:
        return [], -np.inf

    scores_bwd = list()

    X_bwd = [list(i) for i in combinations(X_subsets,
                                           len(X_subsets) - 1)]

    for i in X_bwd:
        scores_bwd.append(eval_subsets(i))
    idx = np.argmax(scores_bwd)
    return X_bwd[idx], scores_bwd[idx]
```

In [29]:

```
'''
코드는 전진 선택법을 사용하여 변수를 선택하는
과정을 구현합니다.

1. `X_subsets`와 `scores` 리스트를 초기화합니다.
   `X_subsets`는 선택된 변수 조합을 저장하고,
   `scores`는 각 변수 조합에 대한 성능을 저장합니다.
2. `X_all`에는 모든 변수의 인덱스가 포함되어 있습니다.
3. `while` 루프는 선택된 변수 조합이 모든 변수를
   포함하거나 성능이 개선되지 않을 때까지 계속됩니다.
4. `forward_select` 함수를 호출하여 현재 변수 조합에서
   가장 성능이 개선되는 변수를 선택합니다.
5. 선택된 변수와 해당 성능을 출력합니다.
6. 만약 이전 단계와 비교하여 성능이 개선되지 않으면
   루프를 종료합니다.
7. 개선된 성능과 변수 조합을 `scores`와
```

`X\_subsets`에 추가합니다.

8. 모든 변수가 선택된 경우 또는 더 이상 선택할 변수가 없는 경우 루프를 종료합니다.
9. 마지막으로, 최종 선택된 변수 조합의 성능과 변수 이름을 출력합니다.

이를 통해 전진 선택법을 사용하여 최적의 변수 조합을 찾고, 해당 변수 조합의 성능과 변수 이름을 확인할 수 있습니다.
'''

전진선택법을 해봅니다.

각 단계에 선택된 속성들을 남기기 위한 리스트 입니다.

X_subsets = [[]]

scores = [-np.inf]

X_all = np.arange(0, 4)

while True:

 X_best, score = forward_select(X_all, X_subsets[-1])

현 단계에서 가장 좋은 속성 조합을 출력합니다.

 print(X_best, score)

성능에 개선이 없으면 종료합니다.

 if score <= scores[-1]:

 break

 scores.append(score)

 X_subsets.append(X_best)

모두 선택되어 더 이상 선택할 변수가 없을 경우 종료합니다.

 if len(X_best) == len(X_all):

 break

scores[-1], X_subsets[-1], \

 [iris['feature_names'][i] for i in X_subsets[-1]]

[3] 0.9583333333333334

[3, 2] 0.9583333333333333

```
Out[29]: (0.9583333333333334, [3], ['petal width (cm)'])
```

```
In [30]: '''
코드는 후진 제거법을 사용하여 변수를 선택하는 과정을 구현합니다.
```

1. `X\_all`에는 모든 변수의 인덱스가 포함되어 있습니다.
2. 초기 변수 조합으로는 모든 변수가 선택된 상태를 의미하는
`X\_subsets`와 해당 성능을 `scores` 리스트에 저장합니다.
3. `while` 루프는 선택된 변수 조합이 하나의 변수만 남거나
성능이 개선되지 않을 때까지 계속됩니다.
4. `backward\_eliminate` 함수를 호출하여 현재 변수 조합에서
성능이 가장 낮은 변수를 제거합니다.
5. 제거된 변수와 해당 성능을 출력합니다.
6. 만약 이전 단계와 비교하여 성능이 개선되지 않으면
루프를 종료합니다.
7. 개선된 성능과 변수 조합을 `scores`와
`X\_subsets`에 추가합니다.
8. 제외할 변수가 더 이상 없는 경우 또는
한 변수만 남은 경우 루프를 종료합니다.
9. 마지막으로, 최종 선택된 변수 조합의 성능과
변수 이름을 출력합니다.

이를 통해 후진 제거법을 사용하여 최적의 변수 조합을 찾고,
해당 변수 조합의 성능과 변수 이름을 확인할 수 있습니다.

후진제거법을 해봅니다.

```
X_all = np.arange(0, 4)
```

```
X_subsets = [X_all]
```

```
scores = [eval_subsets(X_all)]
```

```
while True:
```

```
    X_best, score = backward_eliminate(X_subsets[-1])
```

```
    print(X_best, score)
```

```
    if score <= scores[-1]:
```

```
        break
```

```
    scores.append(score)
```

```
    X_subsets.append(X_best)
```

```
# 더 이상 제외할 변수가 없으면 종료합니다.
```

```
if len(X_best) == 1:
    break
```

```
scores[-1], X_subsets[-1], \
    [iris['feature_names'][i] for i in X_subsets[-1]]
```

```
[0, 2, 3] 0.9583333333333333
```

```
[2, 3] 0.9583333333333333
```

```
Out[30]: (0.9583333333333333,
         [0, 2, 3],
         ['sepal length (cm)', 'petal length (cm)', 'petal width (cm)'])
```

```
In [31]: '''
코드는 단계적 전진선택법(Stepwise Forward Selection)을
구현합니다.
```

1. 초기 변수 조합으로는 빈 리스트를 사용하고,
해당 성능을 음의 무한대로 설정한 후 변수 조합과 성능을
각각 저장하는 `X\_subsets`와 `scores` 리스트를 초기화합니다.
2. `while` 루프는 선택된 변수 조합이 모든 변수를 포함하거나
성능이 개선되지 않을 때까지 계속됩니다.
3. `forward\_select` 함수를 호출하여 현재 변수 조합에서
가장 성능이 개선되는 변수를 선택합니다.
4. 선택된 변수 조합과 해당 성능을 출력합니다.
5. 만약 이전 단계와 비교하여 성능이 개선되지 않으면
루프를 종료합니다.
6. 후진 제거를 수행합니다. 이는 선택된 변수를 제거하여
성능을 확인하는 과정입니다.
7. 후진 제거의 효과가 있으면, 제거된 변수의 조합과
해당 성능을 출력하고, 변수 조합과 성능을 갱신합니다.
8. 최종 선택된 변수 조합의 성능과 변수 이름을 출력합니다.

```
이를 통해 단계적 전진선택법을 통해 최적의 변수 조합을 찾고,
해당 변수 조합의 성능과 변수 이름을 확인할 수 있습니다.
'''
```

```
# 단계적 전진선택법
```

```
X_subsets = [[]]
```

```
scores = [-np.inf]
```

```

X_all = np.arange(0, 4)

while True:
    clf_lr = make_pipeline(StandardScaler(),
                           LogisticRegression(solver='lbfgs',
                                                multi_class='multinomial'))
    X_best, score = forward_select(X_all, X_subsets[-1])
    print(X_best, score)

    # 성능에 개선이 없다면 종료 시킵니다.
    if score <= scores[-1]:
        break

    # 후진 제거를 합니다.
    X_bwd, bwd_score = backward_eliminate(X_best)

    # 후진 제거의 효과가 있다면 제거한 리스트를 선택합니다.
    if score < bwd_score:
        X_best = X_bwd
        score = bwd_score
        print("B.E.: {} {}".format(bwd_score, X_bwd))

    scores.append(score)
    X_subsets.append(X_best)

    if len(X_best) == len(X_all):
        break

scores[-1], X_subsets[-1],
[iris['feature_names'][i] for i in X_subsets[-1]]

```

```
[3] 0.9583333333333334
```

```
[3, 2] 0.9583333333333333
```

```
Out[31]: (0.9583333333333334, [3], ['petal width (cm)'])
```

```
In [32]: '''
```

코드는 단계적 후진제거법(Stepwise Backward Elimination)을 구현합니다.

1. 초기 변수 조합으로는 모든 변수가 선택된 상태를 의미하는

- 1. ``X_all``을 사용하고, 해당 성능을 평가하여 ``scores`` 리스트에 저장합니다.
- 2. ``while`` 루프는 선택된 변수 조합이 하나의 변수만 남거나 성능이 개선되지 않을 때까지 계속됩니다.
- 3. ``backward_eliminate`` 함수를 호출하여 현재 변수 조합에서 성능이 가장 낮은 변수를 제거합니다.
- 4. 제거된 변수 조합과 해당 성능을 출력합니다.
- 5. 만약 이전 단계와 비교하여 성능이 개선되지 않으면 루프를 종료합니다.
- 6. 전진 선택을 수행합니다. 이는 선택된 변수를 추가하여 성능을 확인하는 과정입니다.
- 7. 전진 선택의 효과가 있으면, 선택된 변수의 조합과 해당 성능을 출력하고, 변수 조합과 성능을 갱신합니다.
- 8. 최종 선택된 변수 조합의 성능과 변수 이름을 출력합니다.

이를 통해 단계적 후진제거법을 통해 최적의 변수 조합을 찾고, 해당 변수 조합의 성능과 변수 이름을 확인할 수 있습니다.

'''

단계적 후진제거법

`X_all = np.arange(0, 4)`

`X_subsets = [X_all]`

`scores = [eval_subsets(X_all)]`

while True:

`X_best, score = backward_eliminate(X_subsets[-1])`

`print(X_best, score)`

성능에 개선이 없다면 종료 시킵니다.

if `score <= scores[-1]:`

break

`X_fwd, fwd_score = forward_select(X_all, X_best)`

전진 선택의 효과가 있다면 전진 선택한 리스트를 선택합니다.

if `score < fwd_score:`

`X_best = X_fwd`

`score = fwd_score`

`print("F.S.: {} {}".format(fwd_score, X_fwd))`

`scores.append(score)`

```

X_subsets.append(X_best)

if len(scores) == len(X_all) + 1:
    break

scores[-1], X_subsets[-1],
[iris['feature_names'][i] for i in X_subsets[-1]]

```

```
[0, 2, 3] 0.9583333333333333
```

```
[2, 3] 0.9583333333333333
```

```

Out[32]: (0.9583333333333333,
         [0, 2, 3],
         ['sepal length (cm)', 'petal length (cm)', 'petal width (cm)'])

```

```
In [10]: Image('129.png')
```

```
Out[10]: mlxtend.feature_selection.SequentialFeatureSelector
```

(단계적)전진 선택법과 후진 제거법을 제공합니다.

주요 하이퍼파라미터

이름	설명
estimators	기반 모델
k_features	선택할 입력 변수(속성)의 수 'best': 설정시 가장 좋은 성능을 나타낸 변수 만큼 뽑습니다.
forward	True: 전진 선택법 후진 선택법
floating	단계적 수행 여부
scoring	선택의 기준이 되는 성능 지표
cv	검증 방법을 나타내는 Splitter

[Ex. 10]

`mlxtend.feature_selection.SequentialFeatureSelector`를 이용하여
`{X_iris_train, y_iris_train}` 데이터셋을 이용해서
 단계적 전진선택법을 이용한 속성 선택을 합니다.
 성능 지표는 정확도(Accuracy)입니다.

```
In [17]: '''
SequentialFeatureSelector는 mlxtend 라이브러리에서 제공하는
특성 선택 기법 중 하나입니다.
이는 특성 선택을 위해 전진 선택, 후진 제거 등의 방법을 사용하여
최적의 특성 조합을 찾습니다.

1. 전진 선택과 후진 제거:
    SequentialFeatureSelector는 전진 선택법과 후진 제거법을
    결합하여 사용할 수 있습니다.
    이는 전진 선택법으로 시작하여 후진 제거법을 통해
    특성을 선택하거나 제거하여 최적의 특성 조합을 찾습니다.

2. 부동 특성 선택:
    부동 특성 선택은 선택된 특성들 사이에서 최적의
    특성 조합을 찾는 방법입니다.
    이를 통해 선택된 특성들 사이의 관계를 고려하여
    더 나은 특성 조합을 찾을 수 있습니다.

3. 평가 지표 설정:
    사용자가 원하는 평가 지표를 설정하여 특성의 성능을
    평가할 수 있습니다. 예를 들어, 정확도, 정밀도,
    재현율 등의 지표를 사용할 수 있습니다.

4. 교차 검증:
    교차 검증을 통해 특성의 일반화 성능을
    평가할 수 있습니다.
    이를 통해 특성 선택의 안정성을 높일 수 있습니다.

5. 사용자 정의 모델 설정:
    사용자는 SequentialFeatureSelector에 사용할 모델을
    설정할 수 있습니다. 이를 통해 사용자가 원하는 모델을
    사용하여 특성 선택을 수행할 수 있습니다.
'''
```

`SequentialFeatureSelector`를 사용하면 최적의 특성 조합을 자동으로 찾아내고, 이를 통해 모델의 성능을 향상시킬 수 있습니다. 또한, `mlxtend` 라이브러리의 다양한 특성 선택 기법과 함께 사용하여 다양한 실험을 수행할 수 있습니다.

```
from mlxtend.feature_selection
import SequentialFeatureSelector
```

Out[17]: ((3,), 0.975)

In []:

```
'''
코드는 mlxtend 라이브러리의 SequentialFeatureSelector를
사용하여 단계적 전진선택법을 수행하는 예시입니다.

1. `SequentialFeatureSelector`를 초기화합니다.
   이 때, 사용할 기반 모델을 설정합니다.
   여기서는 StandardScaler와 LogisticRegression을 조합한
   파이프라인을 사용합니다.
   LogisticRegression은 solver로 'lbfgs'를,
   multi_class로 'multinomial'을 설정하여
   다중 클래스 분류를 수행합니다.

2. `k_features='best':`
   최적의 특성 수를 선택하도록 지정합니다.
   이는 모든 가능한 특성 수를 대상으로 하여
   최적의 특성 수를 선택합니다.

3. `forward=True`:
   전진 선택을 수행하도록 설정합니다.
   이는 초기에는 하나의 특성부터 시작하여
   최적의 특성 수까지 특성을 추가하는 방식으로 진행됩니다.

4. `floating=True`:
   부동 특성 선택 방법을 사용합니다.
   이는 선택된 특성들 사이에서 최적의 특성 조합을
   찾는데 도움이 됩니다.

5. `scoring='accuracy':`
   평가 지표로 정확도를 사용하도록 지정합니다.
   즉, 각 단계에서 교차 검증을 통해 정확도를
   측정하여 성능을 평가합니다.

6. `cv=cv`:
   교차 검증을 위해 사용할 폴딩 전략을 설정합니다.
'''
```

- 이 예시에서는 이전에 정의한 ``cv``를 사용합니다.
7. ``fit`` 메서드를 호출하여 모델을 학습시킵니다.
이는 모든 가능한 특성 수에 대해 전진 선택을 수행하고, 최적의 특성 수를 찾습니다.
 8. 학습이 완료되면 ``k_feature_idx_`` 속성에 최적의 특성 인덱스가 저장되고, ``k_score_`` 속성에 해당 최적의 특성 조합의 성능이 저장됩니다.

이를 통해 `SequentialFeatureSelector`를 사용하여 최적의 특성 조합을 찾고, 해당 조합의 성능을 확인할 수 있습니다.

```
'''
```

```
'''
```

부동 특성(Floating Feature)은

`Sequential Feature Selection` 방법 중 하나로, 선택된 특성들 사이에서 다시 특성을 추가하거나 제거하는 과정을 말합니다.
전진 선택법과 후진 제거법을 결합하여 사용됩니다.
전진 선택법에서는 초기에는 하나의 특성부터 시작하여 순차적으로 특성을 추가하면서 최적의 특성 조합을 찾습니다.
하지만 특성을 추가하면서 선택된 특성 조합이 최적이라는 보장은 없습니다.
따라서 후진 제거법을 통해 선택된 특성들을 다시 검토하고, 필요에 따라 추가적인 특성을 제거함으로써 더 나은 특성 조합을 찾습니다.

이러한 후진 제거 과정에서 선택된 특성들 사이에서 선택된 특성을 제거하여 성능을 확인하는 것이 후진 제거법입니다.
이때 선택된 특성을 다시 추가할 수도 있고, 이미 선택된 특성을 다시 제거할 수도 있습니다.
이렇게 선택된 특성들 사이에서 추가적인 특성을 조작하는 과정이 부동 특성입니다.

부동 특성은 선택된 특성들 사이에서 최적의 특성 조합을 탐색하는데 도움이 되며, 전체 특성 공간을 보다 효율적으로 탐색할 수 있도록 도와줍니다.
따라서 부동 특성을 사용하면 더 나은 특성 조합을 찾을 수 있습니다.

```
'''
```

단계적 전진선택법을 수행하도록

```
# SequentialFeatureSelector를 설정합니다.
sfs = SequentialFeatureSelector(
    # 기반 모델을 설정해줍니다.
    make_pipeline(StandardScaler(),
                  LogisticRegression(solver='lbfgs',
                                     multi_class='multinomial')),

    k_features='best',
    forward=True,
    floating=True,
    scoring='accuracy',
    cv=cv
)
sfs.fit(X_iris_train, y_iris_train)
sfs.k_feature_idx_, sfs.k_score_
```

In [34]:

```
'''
코드는 후진 제거법을 사용하여 변수를
선택하는 과정을 나타냅니다.

1. `X_all`은 모든 변수의 인덱스를 포함하는 배열입니다.
2. `subsets` 리스트는 선택된 변수 조합을 저장합니다.
   초기에는 모든 변수가 선택된 상태입니다.
3. `scores` 리스트는 각 변수 조합에 대한 성능을 저장합니다.
   초기에는 모든 변수가 선택된 경우의 성능으로 초기화됩니다.
4. `for` 루프는 변수의 개수가 1개가 될 때까지 반복합니다.
5. 각 반복에서는 현재 선택된 변수 조합을
   `subset` 변수에 저장합니다.
6. Logistic 회귀 모델을
   현재 선택된 변수 조합에 맞춰 학습합니다.
7. 학습된 모델의 가장 영향력이 적은 변수를 찾습니다.
   이 변수를 제외하고 새로운 변수 조합을 만듭니다.
8. 새로운 변수 조합으로 모델을 다시 학습하고,
   성능을 측정합니다.
9. 이전 성능과 비교하여 성능이 개선되었는지 확인합니다.
10. 성능이 개선되지 않았다면
    더 이상 변수를 제거하지 않고 종료합니다.
11. 최종적으로 선택된 변수 조합과 해당 성능을 출력합니다.

이를 통해 후진 제거법을 통해 최적의 변수 조합을 탐색하고,
해당 조합의 성능을 확인할 수 있습니다.
'''
```

```

'''
X_all = np.arange(0, 4)
subsets = [X_all]
scores = [eval_subsets(X_all)]

for i in range(len(X_all) - 1):
    subset = subsets[-1]

    clf_lr.fit(X_iris_train[:, subset], y_iris_train)

    worst_idx = np.argmin(np.mean(np.abs(clf_lr[1].coef_),
                                     axis=0))

    subset = [i for i in subset if i != subset[worst_idx]]
    score = eval_subsets(subset)

    print(subset, score)

    if score >= scores[-1]:
        scores.append(score)
        subsets.append(subset)
    else:
        break
subsets[-1], scores[-1]

```

[1, 2, 3] 0.9416666666666667

Out[34]: (array([0, 1, 2, 3]), 0.95)

In [18]: Image('130.png')

Out[18]: 장단점 비교

방법	장점	단점
Filter 방법	계산 비용이 낮고 빠릅니다. 대규모 데이터셋에도 적용하기 쉽습니다. 머신러닝 모델과 독립적으로 작동하므로 어떤 모델이든 적용 가능합니다.	모델의 상호작용을 고려하지 않고 속성을 개별적으로 평가하기 때문에, 최적의 조합을 고려하지 않을 수 있습니다.
Wrapper 방법	속성 간 상호작용을 고려할 수 있어 더 좋은 성능을 얻을 수 있습니다.	계산 비용이 매우 높습니다.

In []: